

Effort-Aware semi-Supervised just-in-Time defect prediction

Weiwei Li^a, Wenzhou Zhang^b, Xiuyi Jia^{b,*}, Zhiqiu Huang^c

^a College of Astronautics, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, China

^b School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing 210094, China

^c College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, China

ARTICLE INFO

Keywords:

Defect prediction
Just-in-time
Tri-training
Effort-aware

ABSTRACT

Context: Software defect prediction is an important technique that can help practitioners allocate their quality assurance efforts. In recent years, just-in-time (JIT) defect prediction has attracted considerable interest, as it enables developers to identify risky changes at check-in time.

Objective: Many studies have conducted research from supervised and unsupervised perspectives. A model that does not rely on label information would be preferred. However, the performance of unsupervised models proposed by previous studies in the classification scenario was unsatisfactory due to the lack of supervised information. Furthermore, most supervised models fail to outperform simple unsupervised models in the ranking scenario. To overcome this weakness, we conduct research from the semi-supervised perspective that only requires a small quantity of labeled data for training.

Method: In this paper, we propose a semi-supervised model for JIT defect prediction named Effort-Aware Tri-Training (EATT), which is an effort-aware method using a greedy strategy to rank changes. We compare EATT with the state-of-the-art supervised and unsupervised models with respect to different labeled rate.

Results: The experimental results on six open-source projects demonstrate that EATT outperforms existing supervised and unsupervised models for effort-aware JIT defect prediction, and has similar or superior performance in classifying defect-inducing changes.

Conclusion: The results show that EATT can not only achieve high classification accuracy as supervised models, but also offer more practical value than other compared models from the perspective of the effort needed to review changes.

1. Introduction

To produce high-quality software, developers have to expend substantial effort testing and debugging it. Software defect prediction [1] can help developers effectively save testing time and reduce the cost of software development by inferring code segments that may contain defects. In recent years, defect prediction at the change level has drawn considerable interest. This approach to software quality assurance was first proposed by Mockus and Weiss [2] and was referred to as just-in-time (JIT) defect prediction by Kamei et al. [3].

In contrast to defect prediction studies focused on predicting defects at coarse (e.g., file, package, or module) granularity levels [4–7], JIT defect prediction tries to identify defect-prone (“risky”) software changes, the data for which is collected by combining information extracted from the version archive (such as Git) with bug reports. There are many advantages of such prediction compared with traditional software defect

prediction, which can be summarized as the following three points, as stated in Ref. [3]. (1) Predictions can be associated with small areas of code segments and require less effort than the predictions with coarser granularity. (2) Predictions can be easily associated with the developers who performed the corresponding changes. (3) Predictions can be obtained when changes are submitted for integration and the design decisions are still fresh in the minds of developers. Furthermore, Kamei et al. proposed effort-aware JIT defect prediction using lines of code (LOC) to indicate the amount of effort required to review changes. This approach is more practical, as it could help identify more defect-inducing changes at the same inspection cost.

Previous JIT defect prediction studies can be divided into two categories: supervised and unsupervised models. Many supervised strategies have been proposed for predicting risky changes, and a large number of supervised models have hence been developed, including linear regression [3], logistic regression [8], ensemble learning [9,10] and

* Corresponding author.

E-mail addresses: liweiwei@nuaa.edu.cn (W. Li), wenzhou2671@163.com (W. Zhang), jiaxy@njjust.edu.cn (X. Jia), zqhuang@nuaa.edu.cn (Z. Huang).

the others [11,12]. It has been reported that these supervised learning-based models offer a promising prediction performance when classifying defect-inducing changes.

However, the acquisition of labeled data (i.e., identifying whether a change will induce defect or not) often requires skilled human agent and a long labeling process, which is usually quite expensive and time-consuming. Furthermore, previous studies [9,13] have leveraged simple unsupervised models for effort-aware JIT defect prediction and observed that many of them could outperform a supervised model in terms of ACC^1 and P_{opt}^2 , which indicates that simple unsupervised models could detect more defect-inducing changes than supervised models with the same inspection effort. However, Ref. [8] revealed shortcomings of these unsupervised models, including many highly ranked changes being false alarms and models' performance being significantly inferior in terms of the F1 score.

Semi-supervised methods can produce considerable improvement in learning accuracy with a small amount of labeled data in conjunction with abundant unlabeled data [14,15]. Many studies have performed defect prediction at coarser granularity using semi-supervised models [16–27], which shows that such methods can have great practical value in traditional defect prediction tasks. Few study [28] has applied semi-supervised methods for JIT defect prediction, and little is known in the context of effort-aware JIT defect prediction.

To verify the predictive effectiveness of semi-supervised models for JIT defect prediction, we leverage the tri-training [29] method to exploit unlabeled data and propose an effort-aware semi-supervised model named effort-aware tri-training (EATT). Specifically, (1) EATT uses an effort-aware indicator to produce the final hypothesis; (2) instead of subsampling unlabeled data randomly, EATT takes the predicted confidence as well as the review budget into account, and (3) EATT uses a greedy strategy in the unit code inspection effort to rank changes according to their tendency to be defect-prone.

The main contributions of this paper are as follows:

- (1) We investigate predictive effectiveness of sample-based semi-supervised models in effort-aware JIT defect prediction and propose an effort-aware tri-training method for semi-supervised JIT defect prediction.
- (2) We perform an in-depth evaluation of existing supervised and unsupervised models with cross-validation on various labeled-rate-based data.
- (3) We compare EATT with several supervised, unsupervised and semi-supervised methods using various evaluation indicators. The experimental results show that EATT can significantly improve the performance of effort-aware JIT defect prediction even if the labeled data are insufficient.

The rest of this paper is organized as follows. Section 2 introduces the related work on defect prediction. Section 3 describes the over-all framework of JIT defect prediction and presents the details of our approach. Section 4 provides the experimental setup and the research questions in our study. Section 5 gives the answers of these questions and reports the experimental results in detail. Section 6 discusses the results and implications. Section 7 analyzes the threats to the validity of our study. Section 8 concludes this paper.

2. Related work

In this section, we will introduce the related work on JIT defect prediction and some semi-supervised learning applications in traditional defect prediction.

2.1. Just-in-Time defect prediction

Mockus and Weiss [2] are the first to study change-level defect prediction. They used some change metrics such as the number of subsystems touched, the number of files modified, the number lines of added code, and the number of modification requests to predict the risk of defect-inducing changes, which led to an increasing interest in predicting defect at change level. Sliwerski et al. [30] found defect-fixing changes and changes done on Fridays have a higher chance of introducing defects according to the study in the Mozilla and Eclipse open source projects. Eyolfson et al. [31] performed study on the Linux kernel and PostgreSQL and found that changes committed between midnight and 4AM were more likely to be defect-inducing. Yin et al. [32] found that around 14.2% to 24.8% bug-fixing changes are incorrect by studying in several open-source software systems. Kim et al. [33] used numerous features extracted from various sources to build prediction models. Their results showed that the prediction models can obtain 78% accuracy and 60% recall. Kamei et al. [3] performed a large-scale empirical study on six open source projects and five commercial projects. At the same time, they took the effort required to inspect the modified code for a change into account. Their results showed that they can predict defect-inducing changes with 68% accuracy and 64% recall. Kamei et al. [34] explored cross-project models in the context of JIT prediction, but the JIT models rarely performed well in a cross-project context. Yang et al. [13] found that many simple unsupervised models could outperform supervised models when ranking defect-prone changes. Fu et al. [12] improved the related unsupervised models by pruning weaker predictors away. Huang et al. [8,35] found the unsupervised model required developers to inspect numerous changes and proposed an improved supervised model. Yang et al. [9,36] leveraged deep learning techniques and ensemble learning to predict defect-prone changes. Liu et al. [37] found that code churn is a neglected metric and built unsupervised model based on it. Chen et al. [11] proposed a multi-objective optimization based supervised method MULTI. Young et al. [10] reported a replicated experiment and an extension comparing with ensemble learning. Qiao et al. [38] proposed a deep learning based approach. Syed [28] investigated the performance of various well known classifiers and self-training algorithm. Catolino et al. [39] investigated the performance in a mobile environment. Kondo et al. [40] proposed “context metrics” to measure the information around the changed lines. Fan et al. [41] investigated the impact of the mislabeled changes by different SZZ variants on the performance and interpretation of JIT defect prediction models.

2.2. Semi-Supervised learning in traditional defect prediction

Semi-supervised model is a kind of machine learning technique trained by utilizing a few labeled and abundant unlabeled data. In view of semi-supervised approaches' little reliance on labeled data, their predictive effect for software fault prediction has gained considerable researchers' interest and many approaches have been proposed recently. Generally, semi-supervised methods for defect prediction can be typically classified into following categories: expectation maximization [26], naive bayesian algorithm [17], constraint-based semi-supervised clustering [16,25], sample-based approaches [18–21,24], labeled propagation [27], and preprocessing strategy [19,22,23]. These studies show that semi-supervised methods are of practical value in predicting defect at coarser granularity.

Although semi-supervised models have been extensively studied in traditional defect prediction, there are few studies that have conducted research for effort-aware JIT defect prediction. In addition, existing works [13,37] reported that the simple unsupervised models have a prediction performance superior to most supervised model. To this end, we investigate whether semi-supervised models are still effective when making prediction at change level and performing under effort-aware evaluation.

¹ ACC denotes the recall of defect-inducing changes when using 20% of the entire effort to inspect the top ranked changes.

² P_{opt} is the normalized version of the effort-aware performance indicator based on the concept of the “code-churn-based” Alberg diagram. More details could be found in Section 4.5.

From previous studies [29,42–44], the unlabeled data could be helpful for learning if they are exploited properly and many strategies have been proposed to exploit the disagreements among learners during semi-supervised learning process. Some studies have performed sample-based semi-supervised methods to predict defect-prone models in traditional quality assurance. Li et al. [20] proposed two sample-based approaches CoForest and ACoForest for software defect prediction. Their methods perform sample selection by random sampling with a semi-supervised learner and active sampling with an active semi-supervised learner. Lu et al. [21] investigated an iterative semi-supervised software defect prediction approach and used different size of labeled modules in the project to track the performance.

In summary, all these studies show that it is worth investigating the practical value of sample-based semi-supervised methods for JIT defect prediction. Since the semi-supervised methods only need a few labeled data, it will be more desirable for application while they still have good performances.

3. Methodology

In this section, we introduce the overall framework of just-in-time defect prediction first. Then, we describe the tri-training method that is used as the semi-supervised framework of our proposed model. Finally, the details of our model are presented.

3.1. Preprocessing

JIT defect prediction tries to predict whether or not a committed change is defect-inducing. According to the study of Kamei et al. [3], JIT software defect prediction mainly involves four steps: (1) extracting committed data for each change from the CVS repositories of the projects and combining it with bug reports; (2) extracting metrics (i.e., features to represent a change) from each change; (3) training a prediction model based on the features and labels of the training set; and (4) for new changes, using the trained model to predict whether these changes will introduce new bugs after performing the same feature extraction steps.

There are 14 metrics derived from the source control repository data of a project, as shown in Table 1, that are used as the independent variables in this study. The dependent variable is a binary variable (0 or 1) that indicates a code change is defect-inducing or defect-free. The extracted datasets are relatively imbalanced, and most change measures are highly skewed. We use a method similar to that of Ref. [3] to deal with these problems. First, we apply a standard logarithmic transformation to each measure. Then, the training data is resampled by deleting the majority class's instances randomly. After that, defect prediction models can be trained on the new change datasets.

For effort-aware JIT defect prediction, the effort value is measured with the size of change (i.e., the total number of modified lines) [3]. The

reason for using this value is that adding or deleting many lines of code for a change usually tends to require more time to validate than adding or deleting a fewer lines. This type of prediction offers more practical results by considering the effort needed for software quality assurance. In our work, we follow the previous studies [3,11,13] and only use 12 metrics (excluding LA and LD) to build a semi-supervised model, while lines of code added (LA) and lines of code deleted (LD) will be used to represent the effort value in predicting the defect-proneness of changes. We propose an effort-aware tri-training (EATT) method for just-in-time defect prediction, and the overall framework is presented in Fig. 1.

3.2. Tri-Training

The tri-training method that tries to exploit unlabeled data using three classifiers was proposed by Zhou and Li [29]. This setting tackles the problem of determining how to effectively select unlabeled data and when to produce the final hypothesis. In contrast to co-training [42], the tri-training method only needs a single view of the dataset. At the same time, different learning algorithms are of more benefit to learning if each classifier can partition the feature space into a number of equivalent classes [43]. Hence, we use different classifiers with the tri-training method to analyze unlabeled data. Such an approach can also be viewed as an ensemble method. The initial classifiers are trained using the original labeled data. Afterwards, in each tri-training round, some unlabeled data will be selected randomly as labeled data with the same label predicted by two classifiers. By combining the new labeled data with original labeled data, the third classifier will be trained. This process will not stop until the stopping criterion is satisfied (i.e., all three classifiers do not change any more). Finally, the prediction results of three classifiers will be combined by majority voting. Algorithm 1 shows the pseudo-code of the simplified tri-training method, and more details can be found in Ref. [29].

Algorithm 1: Pseudo code of simplified tri-training.

Input: Training set $L = \{x_i, y_i\}_{i=1}^l$, $U = \{x_i\}_{i=1}^u$;

Three classifiers h_1, h_2, h_3 ;

Output: Ensemble h using majority vote;

```

1 for  $i = 1, 2, 3$  do
2   | Train  $h_i$  on  $L$ ;
3 end
4 while any of  $\{h_1, h_2, h_3\}$  changes do
5   for  $i = 1, 2, 3$  do
6     |  $L_i = \emptyset$ ;
7     | for  $x \in U$  do
8       | if  $h_i(x) = h_k(x)(j, k \neq i)$  then
9         | |  $L_i = L_i \cup (x, h_j(x))$ ;
10      | end
11    | end
12  end
13 for  $i = 1, 2, 3$  do
14   | Train  $h_i$  on  $L \cup L_i$ ;
15 end
16 end

```

Table 1
Summary of change measures.

Name	Definition
NS	Number of modified subsystems [2]
ND	Number of modified directories [2]
NF	Number of modified files [45]
Entropy	Distribution of modified code across each file [5,46]
LA	Lines of code added [47,48]
LD	Lines of code deleted [47,48]
LT	Lines of code in a file before the change [49]
FIX	Whether or not the change is a defect fix [50,51]
NDEV	The number of developers that changed the modified files [52]
AGE	The average time interval between the last and the current change [53]
NUC	The number of unique changes to the modified files [5,46]
EXP	Developer experience [2]
REXP	Recent developer experience [2]
SEXP	Developer experience on a subsystem [2]

3.3. Effort-Aware tri-Training

Although traditional semi-supervised methods have been used to obtain impressive results, they are unsuitable for effort-aware defect prediction, as such methods are designed for classification tasks. Due to this restriction, we propose an effort-aware tri-training (EATT) method for JIT defect prediction.

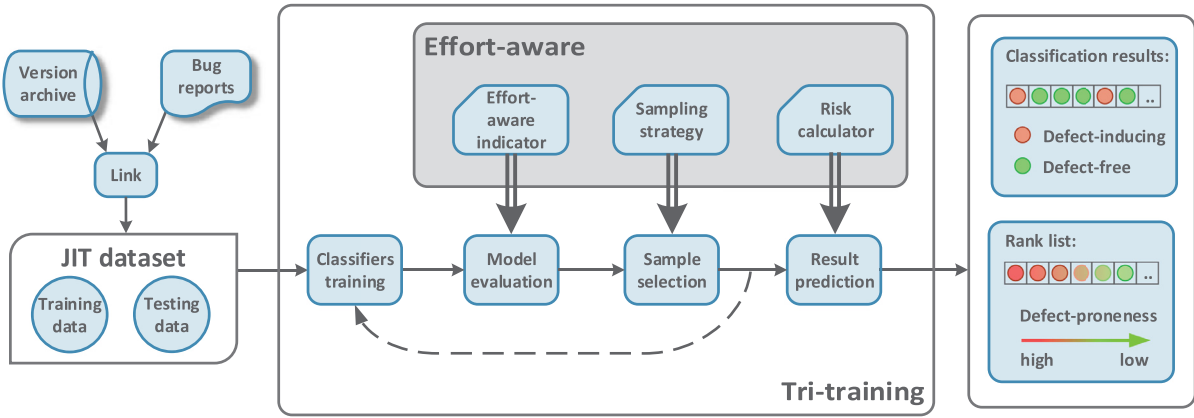


Fig. 1. Illustration of EATT for JIT defect prediction. (1) The JIT dataset is extracted from the combination of version archive and bug reports. (2) Then our semi-supervised model is trained by utilizing the EATT method. (3) Finally, the trained model outputs classification or ranking results.

3.3.1. Problem formulation

Let L denote labeled software changes (i.e., whether or not these changes are defect-inducing is already known) and U denote unlabeled software changes. Let X denote the new changes whose labels we try to predict and function $\text{Effort}(x)$ denote the amount of effort required by change x ($x \in X$). Assuming that there are three classifiers h_1, h_2 and h_3 , EATT tries to analyze unlabeled changes by applying these classifiers and taking the inspection effort into account.

3.3.2. Model evaluation

Zhou and Li [29] utilized the classification error rate derived from the combination of two classifiers to evaluate the performance of a learner. To discover more buggy changes given a lower inspection budget, the recall of defect-inducing changes with fixed effort (equal to 20% of the total available effort) is used to produce the final hypothesis in our work. Although the classification accuracy may decrease, the EATT method could detect more defect-inducing changes by ranking them based on their defect density. The specific steps are illustrated with the following example. Let $L'_3 \subseteq L$ denote the change set such that the predictions of h_1 and h_2 on L are the same. The classification error e_3 with respect to h_3 is estimated by dividing the number of incorrect predictions by the number of consistently predicted changes $|L'_3|$. The effort-aware indicator ACC_3 is estimated by dividing the number of found defects with 20 percent effort by the number of actual defect-inducing changes in L'_3 . The change set L'_3 will be used as a new labeled dataset for training in the next round only if ACC_3 in the current round is greater than that in the preceding round.

3.3.3. Sample select

Assume that the current iteration is the t -th round during training. Let e^t and $|L^t|$ denote the classification error rate and the number of changes introduced into the labeled data in the t -th round, respectively. $L' \subseteq U$ represents the set of changes for which two classifiers have same prediction results. According to the study of [29,54], as long as the number of samples introduced per round satisfies $|L'| < e^{t-1}|L^{t-1}|/e^t$, the negative impact of introducing noise can be compensated by the addition of a sufficient number of labeled examples. In a practical learning process, $|L'|$ is usually larger than $e^{t-1}|L^{t-1}|/e^t$. We prefer to select the changes with higher confidence and extreme effort instead of selecting randomly as was done by previous studies [13,37]. Note that these selected changes will not be put into the original labeled change set (i.e., L). This approach ensures that the introduced noise can be maintained at a lower level and the changes with an increasing confidence will be incorporated into the training data during the iteration process.

3.3.4. Result prediction

To predict whether or not a new change x will introduce a defect, we also use the majority voting strategy. Considering the effort-aware defect prediction, the risk value $R(x)$ is calculated as follows:

$$R(x) = \frac{\sum_i p_i(x)}{\text{Effort}(x)}, \quad (1)$$

where $p_i(x)$ denotes the probability of the change being defect-inducing, as predicted by h_i . Adding the probabilities predicted by three classifiers allows the confidence of all three classifiers to be reflected in the rank list. This form of computation views the tendency to be defect-prone at unit inspection effort as the risk value of changes, and the riskier changes have the defect-inducing inclination.

The pseudo-code of the EATT is shown in Algorithm 2.

Algorithm 2: Effort-Aware Tri-Training for JIT Defect Prediction.

Input: Training set of labeled changes

$L = \{x_i, y_i\}_{i=1}^l$;

Training set of unlabeled changes $U = \{x_i\}_{i=1}^u$;

Testing set of new changes $X = \{x_i\}_{i=1}^n$;

Effort required to inspect all these changes

$E = \{\text{Effort}(x_i)\}_{i=1}^{l+u+n}$;

Three classifiers h_1, h_2, h_3 ;

Output: Rank list sorted by the risk value of each change $x \in X$.

- 1 Train $\{h_1, h_2, h_3\}$ on labeled training set L ;
 - 2 Initialize the recall with a fix effort ACC ; the classification error e ; the number of selected changes l ;
 - 3 **while** any of $\{h_1, h_2, h_3\}$ changes **do**
 - 4 Choose one combination of any two classifiers.
 - 5 Compute e and ACC by evaluating the performance on the set of changes where two classifiers make the same prediction on L ;
 - 6 **if** ACC is greater than that in the last round **then**
 - 7 Get the set of changes L' where the corresponding two classifiers make the same prediction on U ;
 - 8 Compute the number of changes to be selected;
 - 9 Subsample the L' by selecting high confident changes;
 - 10 **end**
 - 11 Update ACC , e and l ;
 - 12 Retrain the remaining classifier on the new labeled data $L \cup L'$;
 - 13 **end**
 - 14 Compute the risk value $R(x)$ of each change $x \in X$ by Eq.(1).
-

Table 2
Statistics of the studied projects.

Project	Period	#changes	%buggy
BUG	1998/08/26-2006/12/16	4620	36%
COL	2002/11/25-2006/07/27	4620	30%
JDT	2001/05/02-2007/12/31	35,386	14%
PLA	2000/01/01-2006/12/17	64,250	14%
MOZ	2001/05/02-2007/12/31	98,275	5%
POS	1996/07/09-2010/05/15	20,431	15%

4. Experimental setup

In this section, we first describe the statistics of datasets used in our experiments. Then, we introduce the baseline models compared in our study. Next, we present the evaluation methods used to analyze the effectiveness of EATT. After that, we describe the indicators used to evaluate the performance of defect prediction models. Finally, the research questions are presented.

4.1. Datasets

In this work, we perform semi-supervised defect prediction experiments on six large open-source projects shared by Kamei et al. [3,34], including Bugzilla (BUG), Columba (COL), Eclipse JDT (JDT), Eclipse Platform (PLA), Mozilla (MOZ) and PostgreSQL (POS). These six projects are large long-lived projects that cover a wide range of domains and sizes. Each instance corresponds to a change committed when the code is submitted to the version control system. Table 2 summarizes the statistics of the studied projects, including the time period of each project, the total number of changes and the percentage of buggy changes. It is observed that the positive instances (defect-inducing changes) only account for a small part of the total number of instances. Obtaining labeled changes is very expensive, so it is necessary to study if satisfactory performance can be attained by semi-supervised learning in JIT defect prediction.

4.2. Baseline models

We compare EATT with three kinds of defect prediction models, namely, supervised, unsupervised and semi-supervised models. The supervised and unsupervised models are state-of-the-art methods for JIT defect prediction, and the semi-supervised ones are sample-based methods of traditional defect prediction.

The supervised models include EALR [3], TLEL [9], CBS [35], OneWay (OW) [12], and MULTI [11], and the unsupervised models include LT [13] and CCUM [37]. EALR is a linear regression-based effort-aware JIT defect prediction model. TLEL applies a two-layer ensemble strategy to build a JIT defect prediction model. CBS is a supervised model that sorts changes after classifying them. OneWay is a supervised model based on simple unsupervised learners. MULTI is a supervised JIT defect prediction model based on multi-objective optimization. The LT model is an unsupervised model that ranks changes according to the measure *LT*. CCUM is built by ranking changes according to code churn ($LA + LD$).

The sample-based semi-supervised methods include CoForest (CF) [20] and FTF [21]. CoForest labels changes based on the random forest and random sampling with a semi-supervised learner. FTF is a semi-supervised method based on self-training.

4.3. Classifier selection

In our experiment, three different classifiers are used as our base models. They are support vector machine, logistic regression and random forest classifier. There are two reasons for selecting these classifiers. First, these classifiers are mature and popular techniques that have been widely used in previous JIT defect prediction studies [3,9,11,35,36].

Second, there is a large disagreement between these classifiers while their performance is comparable, which is thought to be beneficial for semi-supervised learning [44]. Our implementation³ is in *Python*, and all the parameters of classifiers use the default values provided by the *sklearn* package. For a fair comparison, parameters of classifiers used by other baseline models are the same.

4.4. Experiment setting

We perform 10 times 10-fold cross-validation and time-wise cross-validation to examine defect prediction models in our experiments. In particular,

- To perform 10-fold cross-validation 10 times, the dataset is divided into 10 parts, of which nine parts are used as training data, and the remaining part is used for testing. As we try to apply a semi-supervised method to JIT defect prediction, the training data is processed by randomly removing the labels of changes at the specified rate. We keep training instances under rates at 0.1 and 0.2 labeled, while these rates are also used in traditional semi-supervised defect prediction [16,27]. Note that the number of labeled positive samples (i.e., defect-inducing changes) is the same as that of labeled negative samples (i.e., defect-free changes). After that, the training data consists of a small quantity of labeled data and abundant unlabeled data. The final prediction performance is then aggregated across all 100 validation results, and the mean value is reported.
- For time-wise cross-validation, we extend the training dataset by introducing the changes of additional two months as an unlabeled dataset. More specifically, assume that the changes in a project are ranked according to their commit data and grouped into n parts based on their dates (i.e., year and month). We build a model on the labeled dataset that combines the i -th part and the $(i + 1)$ -th part and the unlabeled dataset that combines the $(i + 2)$ -th part and the $(i + 3)$ -th part. Then, the trained model is used to predict the labels of changes in the $(i + 4)$ -th part and the $(i + 5)$ -th part ($1 \leq i \leq n - 5$). The average performance of $n - 5$ values of prediction effectiveness is recorded for comparison.

To examine whether there is a significant difference in the prediction performance between the proposed method and other baseline methods, we apply the Wilcoxon signed-rank test [55] with Benjamini-Hochberg (BH) correction [56] at 95% significant level on two competing models. If the BH corrected p -value reported by Wilcoxon's signed-rank test lower than 0.05, the difference is considered to be significantly different and otherwise not. After that, Cliff's δ [57] will be used to measure the magnitude if statistical test shows a significant difference. By convention, the magnitude of the difference is considered trivial for $|\delta| \leq 0.147$, small for $0.147 \leq |\delta| \leq 0.33$, medium for $0.33 \leq |\delta| \leq 0.474$, and large for $|\delta| \geq 0.474$. A model with positive and non-trivial magnitude ($\delta \geq 0.147$) indicates more practical importance than the other one.

4.5. Performance indicators

Six performance indicators—*precision*, *recall*, *F1*, *AUC*, *ACC* and P_{opt} —are used to evaluate the performance of JIT defect prediction models. These indicators can be classified into effort-aware measures for the ranking scenario and non-effort-aware measures for the classification scenario.

Measures *precision*, *recall* and *F1* are indicators commonly used to evaluate classification performance by many previous studies [3,9,36,37]. All these measures can be calculated based on the confusion matrix shown as Table 3. *Precision* (*PRE*) is calculated as $PRE =$

³ The data and code used in this paper are available at <https://github.com/NJUST-IDAM/EATT>.

Table 3
confusion matrix.

	Predicted buggy	Predicted clean
True buggy	TP	FN
True clean	FP	TN

$\frac{TP}{TP+FP}$, and *recall* (*REC*) is calculated as $REC = \frac{TP}{TP+FN}$. *F1* is a harmonic mean of *precision* and *recall* and is calculated as $F1 = \frac{2*REC*PRE}{REC+PRE}$.

AUC is the area under the receiver operating characteristic curve. This curve is plotted in a two-dimensional space with false positive rate as x-coordinate and the true positive rate (recall) as y-coordinate. It is known as a threshold-independent indicator and widely used in traditional defect prediction [1]. The higher *AUC* represents the better performance and the value of 0.5 means the performance of a random predictor.

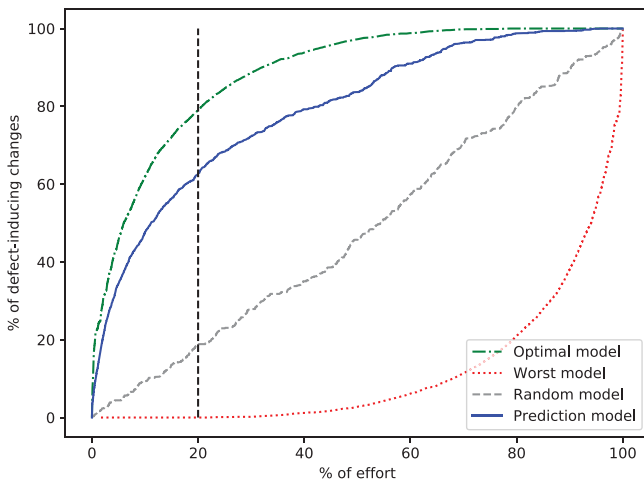
In previous studies [3,11–13,37], *ACC* and P_{opt} were also used to evaluate ranking performance by taking the review effort into account. *ACC* denotes the recall of defect-inducing changes observed when using 20% of the entire effort to inspect the top-ranked changes. P_{opt} is defined as $1 - \Delta_{opt}$, where Δ_{opt} is the area between the effort-based cumulative lift chart of the optimal model and the prediction model. The formula for Δ_{opt} has been normalized by the maximum Δ_{opt} , and the range of P_{opt} is [0, 1]. Fig. 2 illustrates the churn-based cumulative lift chart for different models. The *Optimal* line represents that all change instances are sorted in ascending according to their defect-density while the *Worst* line sets a opposite order. A predictor that can achieve a value of P_{opt} above 0.5 is more effective than a random predictor and display a line over the *Random* one. Formally, the P_{opt} of prediction model *m* can be defined as:

$$P_{opt}(m) = 1 - \frac{Area(Optimal) - Area(m)}{Area(Optimal) - Area(Worst)}. \quad (2)$$

4.6. Research questions

To evaluate the effectiveness of our semi-supervised EATT method, we investigate the following four research questions.

- RQ1: Could EATT obtain better results than those of the supervised models?
- RQ2: How does the performance of EATT compare to that of unsupervised methods?
- RQ3: Does EATT perform better than other sample-based semi-supervised approaches?
- RQ4: How does EATT compare to other methods using time-wise cross-validation?

**Fig. 2.** Code-churn-based Alberg diagram.

5. Experimental results and analysis

In this section, we answer our research questions and report the experimental results.

RQ1: Could EATT obtain better results than those of the supervised models?

To answer this question, we compare EATT with five state-of-the-art supervised methods under different evaluation strategies, as described in Section 4.4. All the supervised and semi-supervised models are trained based on the same labeled data, while the semi-supervised models also utilize the unlabeled data.

Before starting off with the analysis, several forms of result are adopted and their corresponding meaning are as follows:

- The results with **bold** font represent the best one among the group of contrastive experimentation.
- The results with a **red** background indicate that the corresponding baseline model is significantly better than EATT, where the BH corrected *p*-value is less than 0.05 and the magnitude of difference is not trivial according to Cliff's δ . Different shades of red indicate the effectiveness level with light red for small, medium red for medium and deep red for large.
- The results with a **green** background indicate that EATT is significantly better than the runner up model, and light green indicates small effectiveness level, medium green for medium and deep green for large.

According to the results shown in Tables 4 and 5, CBS and TLEL outperform EATT in terms of *recall*. TLEL has larger advantage over EATT in 4/6 the cases at labeled rate 0.1 and 0.2. There is a small reduction in CBS's advantage as the labeled rate increases. The number of red cases decreases from 5 to 2 in Table 4. However, considering the harmonic mean of *precision* and *recall* (i.e., *F1*), these models have no clear advantage anymore. The differences between the mentioned models and EATT are not as large, and EATT even attains the best average *F1* score with respect to 10 times 10-fold cross-validation. One naive explanation for this phenomenon may be that CBS and TLEL have a preference for high *recall*, i.e., predict more changes to be defect-inducing, even as performance in terms of *precision* may decline. The *F1* measure is a tradeoff between *precision* and *recall*, and is usually considered more scientific than a single metric. Besides, according the threshold-independent indicator *AUC*, EATT outperforms the second-placed model significantly in 3/6 the cases at labeled rate 0.1 and 4/6 the cases at labeled rate 0.2. The above analyses suggest that EATT is slightly superior to the supervised models in the classification scenario when the labeled data is insufficient.

For effort-aware JIT defect prediction, EATT outperforms all the supervised models on six datasets in terms of *ACC* and P_{opt} under cross-validation. The magnitudes of difference are all large ($\delta \geq 0.474$), which indicates that EATT performs significantly better in effort-aware JIT defect prediction than all supervised models.

Therefore, we can conclude that with the help of proposed strategy, EATT is comparable to the supervised models in predicting defect-inducing changes and performs significantly better than almost all the supervised models in effort-aware JIT defect prediction.

RQ2: How does the performance of EATT compare to that of the unsupervised method?

To answer this question, we compare EATT with two unsupervised models, namely, LT and CCUM. They are two state-of-the-art unsupervised models that have outperformed the existing supervised models.

Since the two unsupervised models are not specifically trained for predicting probability of inducing a defect under classification scenario, we have not compared them using the indicators *precision*, *recall*, *F1* and *AUC*. A similar consideration also applies when comparing EATT with the supervised models EALR and MULTI.

The unsupervised models LT and CCUM predict the reviewed changes within 20% of the entire effort as defect-inducing in Ref. [37].

Table 4

The performance of the compared semi-supervised models and some supervised models under 10 times 10-fold cross-validation.

	Project	0.1 labeled rate						0.2 labeled rate					
		supervised			semi			supervised			semi		
		CBS	TLEL	OW	CF	FTF	EATT	CBS	TLEL	OW	CF	FTF	EATT
PRE	BUG	.5143	.5162	.3556	.5045	.5761	.5330	.5226	.5319	.3522	.5075	.6037	.5424
	COL	.4804	.4582	.2663	.4326	.5221	.4893	.4828	.4684	.2657	.4434	.5254	.4947
	JDT	.2569	.2436	.1155	.2123	.2871	.2707	.2531	.2501	.1153	.2168	.2980	.2726
	MOZ	.1299	.1275	.0319	.1080	.1711	.1436	.1331	.1321	.0330	.1073	.1751	.1483
	PLA	.2563	.2644	.1172	.2343	.3186	.2702	.2578	.2714	.1157	.2409	.3296	.2716
	POS	.4831	.4502	.1889	.4141	.5409	.4953	.4892	.4575	.1830	.4304	.5534	.5026
	AVG	.3535	.3433	.1792	.3176	.4027	.3670	.3564	.3519	.1775	.3244	.4142	.3720
REC	BUG	.6465	.6219	.4231	.4602	.4320	.6232	.6566	.6447	.4197	.4987	.4667	.6436
	COL	.6252	.6498	.6398	.4591	.4823	.5905	.6193	.6610	.6523	.5077	.5032	.6016
	JDT	.6442	.6579	.5372	.4840	.4735	.6201	.6435	.6665	.5407	.5027	.4849	.6279
	MOZ	.6211	.6880	.2711	.5292	.5470	.6263	.6204	.6984	.3043	.5535	.5636	.6331
	PLA	.7012	.6952	.5010	.5104	.5098	.6951	.6959	.6988	.5049	.5262	.5136	.7018
	POS	.6262	.6574	.4970	.5140	.5220	.6150	.6223	.6679	.5245	.5085	.5347	.6193
	AVG	.6441	.6617	.4782	.4928	.4944	.6284	.6430	.6729	.4911	.5162	.5111	.6379
F1	BUG	.5717	.5625	.3744	.4775	.4905	.5718	.5811	.5816	.3725	.5015	.5242	.5870
	COL	.5414	.5362	.3743	.4429	.4973	.5319	.5410	.5470	.3756	.4714	.5111	.5410
	JDT	.3669	.3553	.1892	.2945	.3563	.3764	.3631	.3635	.1878	.3026	.3686	.3798
	MOZ	.2147	.2150	.0567	.1784	.2600	.2335	.2190	.2222	.0592	.1791	.2669	.2402
	PLA	.3753	.3830	.1892	.3207	.3913	.3890	.3761	.3908	.1877	.3300	.4011	.3915
	POS	.5445	.5339	.2704	.4542	.5297	.5477	.5470	.5427	.2702	.4573	.5430	.5543
	AVG	.4358	.4310	.2424	.3614	.4208	.4417	.4379	.4413	.2422	.3736	.4358	.4490
AUC	BUG	.7135	.7101	.4877	.5972	.6780	.7254	.7195	.7319	.4842	.6168	.6985	.7408
	COL	.7212	.7108	.4073	.5936	.6783	.7172	.7259	.7287	.4228	.6232	.7034	.7310
	JDT	.7167	.7067	.3850	.5921	.6824	.7224	.7180	.7204	.3828	.6164	.6969	.7319
	MOZ	.7629	.7790	.3566	.6461	.7555	.7813	.7652	.7891	.3550	.6679	.7659	.7893
	PLA	.7386	.7470	.3793	.6134	.7209	.7544	.7406	.7559	.3730	.6414	.7310	.7603
	POS	.7707	.7663	.3478	.6394	.7392	.7732	.7730	.7736	.3247	.6668	.7482	.7796
	AVG	.7373	.7366	.3939	.6136	.7091	.7457	.7404	.7499	.3904	.6388	.7240	.7555
ACC	BUG	.5436	.5006	.4231	.5794	.5727	.7632	.5484	.5164	.4275	.6157	.6330	.7594
	COL	.5215	.5217	.6398	.6192	.5867	.7986	.5217	.5386	.6562	.6649	.6594	.8058
	JDT	.5450	.5351	.5372	.6084	.5860	.7449	.5469	.5522	.5487	.6466	.6447	.7540
	MOZ	.4441	.4986	.2711	.5333	.5242	.6239	.4454	.5120	.3149	.5602	.5639	.6265
	PLA	.6144	.5961	.5010	.6449	.6463	.7840	.6121	.6059	.5085	.6783	.6929	.7856
	POS	.4975	.4976	.4970	.5786	.5621	.7175	.4953	.5079	.5255	.6097	.6104	.7239
	AVG	.5277	.5250	.4782	.5940	.5796	.7387	.5283	.5388	.4911	.6292	.6340	.7425
P_{opt}	BUG	.7120	.5953	.7444	.7023	.7077	.9242	.7302	.6183	.7419	.7569	.7819	.9247
	COL	.6219	.5984	.8361	.7182	.6863	.9336	.6194	.6121	.8495	.7706	.7626	.9349
	JDT	.6547	.6156	.7831	.7239	.6945	.8847	.6525	.6271	.7837	.7608	.7486	.8881
	MOZ	.6216	.6165	.6396	.6897	.6751	.8212	.6221	.6283	.6485	.7269	.7211	.8228
	PLA	.7043	.6588	.7691	.7394	.7277	.8993	.7022	.6664	.7715	.7758	.7793	.9005
	POS	.6310	.6018	.7870	.7201	.6934	.9035	.6250	.6122	.8017	.7588	.7531	.9057
	AVG	.6576	.6144	.7599	.7156	.6974	.8944	.6586	.6274	.7661	.7583	.7578	.8961

In this case, the prediction effectiveness values of *recall* and *ACC* are the same because *ACC* is defined as the recall of defect-inducing changes observed when using 20% of the entire effort to inspect the top-ranked changes. The performance of CCUM under classification scenario will be discussed in Section 6.

Table 5 shows the performance of LT and CCUM compared to EATT. In this table, EATT performs better than LT in all cases according to *ACC* and P_{opt} , and the superiority of EATT is clear. Compared with CCUM, EATT is inferior in one case in terms of P_{opt} at the labeled rate of 0.1 and shares similar performance on projects COL and JDT. Although CCUM performs well without using labeled data, EATT can further improve

the performance by using only a small amount of labeled data. Besides, EATT obtains non-trivial magnitude of difference over the best unsupervised model on four projects.

Furthermore, it is worth noting that although EATT only slightly increases the performance according to *ACC* and P_{opt} , it is still comparable to supervised models of the true effort is unavailable, while CCUM relies heavily on the predefined effort (i.e., LOC) as the foundation for sorting, which may not be the same in the real-world application.

Overall, the above analysis shows that EATT is more capable than unsupervised models in practical applications when predicting either defect-inducing changes or the risk value of changes.

Table 5

The ranking performance of the compared unsupervised models and other supervised models under 10 times 10-fold cross-validation.

	Project	0.1 labeled rate					0.2 labeled rate				
		supervised		unsupervised		semi	supervised		unsupervised		semi
		EALR	MULTI	LT	CCUM	EATT	EALR	MULTI	LT	CCUM	EATT
ACC	BUG	.3870	.6960	.4788	.7516	.7632	.3905	.6960	.4744	.7465	.7594
	COL	.3897	.6960	.6036	.7982	.7986	.4031	.6960	.6043	.8048	.8058
	JDT	.1953	.6260	.5583	.7302	.7449	.2167	.6260	.5724	.7404	.7540
	MOZ	.1534	.5110	.3587	.5883	.6239	.1570	.5110	.3649	.5914	.6265
	PLA	.2998	.6840	.5150	.7606	.7840	.2951	.6840	.5150	.7601	.7856
	POS	.2698	.6130	.5231	.6985	.7175	.2749	.6130	.5304	.7044	.7239
	AVG	.2825	.6380	.5063	.7212	.7387	.2896	.6380	.5102	.7246	.7425
P_{opt}	BUG	.6986	.8830	.7516	.9154	.9242	.7139	.8830	.7494	.9140	.9247
	COL	.6003	.8800	.8275	.9339	.9336	.6035	.8800	.8283	.9346	.9349
	JDT	.4918	.8290	.7905	.8781	.8847	.4959	.8290	.7943	.8807	.8881
	MOZ	.4662	.7570	.6559	.8044	.8212	.4680	.7570	.6584	.8051	.8228
	PLA	.5557	.8530	.7735	.8897	.8993	.5538	.8530	.7741	.8900	.9005
	POS	.5126	.8430	.8003	.8950	.9035	.5204	.8430	.8039	.8964	.9057
	AVG	.5542	.8410	.7665	.8861	.8944	.5593	.8410	.7681	.8868	.8961

RQ3: Does EATT perform better than other sample-based semi-supervised approaches?

To answer this question, we compare EATT with CoForest and FTF, which are sample-based semi-supervised models for traditional defect prediction. We use a similar strategy to predict the risk value of defect-inducing changes, i.e., we divide the predicted probability of a change being defect-inducing by the effort required to inspect that change.

According to the results shown in Table 4, FTF has a significant advantage over EATT in terms of *precision* on all projects and *F1* on project MOZ and PLA. However, EATT has a better average *F1* score over six projects and outperforms FTF significantly on about half projects in terms of *AUC*.

An explanation of this phenomenon similar to the reasoning stated in RQ1 is that FTF shows an opposite preference to that of the supervised models CBS and TLEL, and predicts fewer changes to be defect-inducing, which is affected by cut-off threshold greatly.

At the same time, EATT outperforms CoForest in all the cases in the classification scenario.

For effort-aware JIT defect prediction, both CoForest and FTF improve substantially as the labeled rate increases. However, EATT can achieve a high performance even though the labeled data is insufficient. In addition, EATT always significantly outperforms CoForest and FTF according to ACC and P_{opt} by applying different evaluation strategies.

Overall, the above observations show that EATT still outperforms other sample-based semi-supervised models.

RQ4: How does EATT compare to other methods using time-wise cross-validation?

Tables 6 and 7 describe the performance in the classification and ranking scenarios respectively, attained using time-wise cross-validation. Accordingly, we make the following observations. First, according to Table 6, the supervised models CBS and TLEL outperform other models using time-wise cross-validation with respect to *recall*, and also significantly outperform EATT with a non-trivial effect size. CBS achieve higher performance over EATT with respect to *F1* and *AUC* while most cases shares no-trivial difference. These results indicate that EATT has similar performance to CBS and TLEL under time-wise cross-validation.

Second, EATT remains invincible in the ranking scenario. Most of compared models are significantly inferior to EATT. Only a slightly supervised model (MULTI) and an unsupervised model (CCUM) perform similarly to our semi-supervised model.

As only three months' training data is available under time-wise cross-validation, the models are not well trained and considered under-fitting. We deem it necessary to investigate how the number of training data and months contribute to prediction effectiveness.

Table 6

The classification performance of the compared semi-supervised models and some supervised models under time-wise cross-validation.

	Project	supervised			semi		
		CBS	TLEL	OW	CF	FTF	EATT
PRE	BUG	.4806	.4778	.3669	.4233	.4822	.4783
	COL	.4264	.4092	.2379	.3405	.4296	.4405
	JDT	.2410	.2187	.1143	.1837	.2295	.2416
	MOZ	.1233	.1160	.0371	.0888	.1343	.1332
	PLA	.2293	.2327	.1130	.1970	.2593	.2387
	POS	.4204	.4061	.2070	.3368	.4540	.4370
	AVG	.3202	.3101	.1794	.2617	.3315	.3282
REC	BUG	.5846	.5571	.3834	.5175	.3766	.5553
	COL	.6513	.6058	.5054	.5175	.4141	.5760
	JDT	.6019	.5621	.4419	.5267	.3813	.5376
	MOZ	.6328	.6672	.2973	.6203	.5403	.6123
	PLA	.6296	.6006	.4308	.5421	.4078	.5984
	POS	.6231	.6213	.4647	.5844	.4881	.5834
	AVG	.6206	.6024	.4206	.5514	.4347	.5772
F1	BUG	.4998	.4840	.3396	.4354	.3769	.4804
	COL	.4852	.4661	.3069	.3922	.3858	.4707
	JDT	.3292	.3019	.1743	.2607	.2669	.3186
	MOZ	.2015	.1935	.0641	.1526	.2079	.2122
	PLA	.3300	.3279	.1735	.2792	.3037	.3338
	POS	.4849	.4746	.2714	.4112	.4448	.4793
	AVG	.3884	.3747	.2216	.3219	.3310	.3825
AUC	BUG	.6212	.6193	.4692	.5758	.5870	.6213
	COL	.6979	.6750	.3684	.6034	.6461	.6897
	JDT	.6965	.6594	.4034	.5839	.6299	.6766
	MOZ	.7433	.7431	.3682	.6566	.7185	.7526
	PLA	.6983	.6942	.3892	.6133	.6636	.7076
	POS	.7247	.7138	.3881	.6236	.6825	.7191
	AVG	.6970	.6842	.3977	.6094	.6546	.6945

Table 7
The ranking performance of the compared models under time-wise cross-validation.

	Project	supervised					unsupervised		semi-supervised		
		EALR	MULTI	CBS	TLEL	OW	LT	CCUM	CF	FTF	EATT
ACC	BUG	.3002	.5980	.4416	.3967	.3834	.4466	.6398	.5821	.5588	.6437
	COL	.3394	.6860	.4790	.4194	.5054	.4673	.6891	.6260	.5601	.6860
	JDT	.3378	.6250	.4537	.3994	.4419	.4676	.6305	.5808	.5440	.6402
	MOZ	.1911	.5490	.4462	.4661	.2973	.3677	.5879	.5877	.5740	.6127
	PLA	.3080	.6880	.5062	.4650	.4308	.4669	.6931	.6562	.6174	.7100
	POS	.3188	.5920	.4475	.4116	.4647	.4694	.6234	.5932	.5606	.6396
	AVG	.2992	.6230	.4624	.4264	.4206	.4476	.6440	.6043	.5691	.6553
P_{opt}	BUG	.5879	.8380	.6724	.5373	.6667	.7106	.8702	.8090	.7469	.8705
	COL	.5712	.8850	.6567	.5407	.7557	.7331	.8846	.8208	.7301	.8843
	JDT	.5855	.8280	.6273	.5215	.7069	.7135	.8254	.7828	.7094	.8318
	MOZ	.4963	.7860	.6241	.5934	.6402	.6518	.7988	.7938	.7606	.8139
	PLA	.5732	.8590	.6484	.5631	.7146	.7302	.8582	.8209	.7611	.8660
	POS	.5651	.8370	.6342	.5521	.7353	.7343	.8493	.8103	.7500	.8585
	AVG	.5632	.8390	.6438	.5513	.7032	.7122	.8477	.8063	.7430	.8542

Overall, EATT performs similarly to or better than the compared models under both classification and ranking tasks by simulating the real-world development scenarios.

6. Discussion

In the preceding sections, we have shown that our model performs competitively with supervised and unsupervised ones. Additionally, more potential insights can be derived through the analysis. In this section, we not only further confirm the superiority of the proposed model using experimental evidence but also discuss the factors that may substantially affect performance.

6.1. Performance comparison under cross-project scenario

Most recently, many studies also performed JIT defect prediction across different projects [8,9,11,13,34], which constructed models based on one project (i.e., source project) and applied these models to predict changes on another project (i.e., target project). To compare with these works, we apply cross-project prediction setting to the baseline models and report the results of the best supervised, unsupervised and semi-supervised models respectively. The cross-project models are trained on the source project and made predictions on the target project. Due to the randomness introduced when sampling the training data, we repeated the operation 10 times and reported their average values in terms of AUC and P_{opt} . As AUC and P_{opt} are independent of artificial cut-off value (classification threshold 0.5 and inspect budget 0.2), they evaluate the performance of prediction model more scientifically. We adopt them for further discussion and same consideration is invited in Section 6.2, Section 6.3 and Section 6.4. There are 30 prediction effectiveness values and the statistical significance and practical importance are determined by BH-corrected p -values and Cliff's δ as well. At this time, all the labels of training set are available, and the EATT method is degraded into an ensemble learning method based on bagging (bootstrap aggregating) [58].

Table 8 summarizes the mean AUC and P_{opt} scores of the best baseline models and EATT obtained from cross-project prediction.

When predicting the probability of inducing defects, we compared EATT with the methods CBS, TLEL, OneWay, CoForest, FTF and CCUM, which are models that achieve the best performance in terms of AUC (MULTI is not included as the results are not available for us). From the left part of Table 8, EATT obtains the most best cases with 13/30

when making prediction across projects, and TLEL achieves the best average score in terms of AUC . The significant difference between EATT and TLEL are trivial according to Cliff's δ , which indicates that they share similar effectiveness in predicting defect-inducing changes. Besides, other baseline models perform significantly worse than EATT with non-trivial effect sizes. Another point worth emphasizing and sketching the phenomenon is that the scores achieved by OneWay and CCUM is less than 0.5, which indicates a naive model employed random guess strategy could transcend them. The above observation demonstrates the importance of supervised information when predicting defect-inducing changes, even though different projects are involved.

The right part of Table 8 summarizes the performance of the best baseline models including CBS, TLEL, MULTI, CoForest, FTF and CCUM when making prediction on the risk value of changes. It can be seen that EATT achieved the best results in more than half of cases (16/30) in terms of P_{opt} and most methods are significantly worse than EATT except MULTI (The results are obtained from Ref[11]). According to P_{opt} , MULTI has relatively stable performance on different target projects, one of which obtains the best performance when using Columba as the source project. The predicted results over five target projects exhibit the runner-up (EATT algorithm) 2.98%, 6.17%, 16.41%, 6.11%, 4.29% improvement respectively. However, EATT still achieved the best performance when combining with the average score in all cases. According to the Wilcoxon signed-rank test and Cliff's δ , only MULTI can obtain performance similar to EATT, and the rest of models are significantly inferior to the proposed one. Specifically, EATT performs significantly better than CBS, TLEL, FTF, CoForest and CCUM with effect sizes are large, large, medium, small and small, respectively. At the same time, the semi-supervised methods CoForest and FTF based on the proposed ranking strategy achieve significantly better performance than the supervised methods CBS and TLEL, which further prove the effectiveness of proposed strategy.

In conclusion, even making prediction across projects, the proposed method still have competitive performance in either classification or ranking scenarios.

6.2. The impact of different combinations of base classifiers

In previous sections, we use three different base classifiers to train the proposed EATT method including support vector machine (SVM), logistic regression (LR) and random forest (RF). In order to investigate the impact of different combination of base classifiers. We further extend the optional base classifiers by adding multi-layer perceptron (MLP),

Table 8

The performance of the top baseline models under cross-project validation.

Train	Test	AUC							P_{opt}						
		CBS	TLEL	OW	CF	FTF	CCUM	EATT	CBS	TLEL	MULTI	CF	FTF	CCUM	EATT
BUG	COL	.6054	0.6034	0.3698	0.5736	0.5826	0.4313	0.5877	0.5747	0.4610	0.8940	0.9373	0.8954	.9450	0.9433
	JDT	.6119	0.5989	0.4119	0.5861	0.5805	0.4098	0.5929	0.5895	0.4297	.9230	0.8863	0.8318	0.8896	0.8945
	MOZ	0.6503	0.6557	0.3891	0.6336	0.6253	0.3414	.6559	0.6508	0.5557	.9220	0.8173	0.8035	0.8060	0.8227
COL	PLA	.6360	0.6119	0.4152	0.5932	0.5888	0.4189	0.6072	0.6358	0.4480	.9180	0.8874	0.8414	0.8915	0.8946
	POS	.6299	0.5780	0.3675	0.5809	0.5650	0.3743	0.5760	0.5702	0.3359	.9180	0.8969	0.8410	0.9021	0.9048
	BUG	0.5760	0.5901	0.5289	0.5405	0.5552	0.4191	.6016	0.6538	0.5555	.9450	0.9131	0.9035	0.9169	0.9176
JDT	JDT	0.5943	0.6162	0.4476	0.5829	0.5917	0.4098	.6243	0.5967	0.5221	.9450	0.8869	0.8719	0.8896	0.8901
	MOZ	0.6374	0.6673	0.4233	0.6054	0.6198	0.3414	.6689	0.6381	0.6361	.9440	0.8095	0.8056	0.8060	0.8109
	PLA	0.5865	.6196	0.4392	0.5903	0.5916	0.4189	0.6160	0.5383	0.5005	.9460	0.8888	0.8772	0.8915	0.8915
MOZ	POS	0.6374	0.6564	0.4651	0.5993	0.6102	0.3743	.6735	0.6344	0.5106	.9430	0.8999	0.8868	0.9021	0.9042
	BUG	0.6116	.6213	0.4409	0.5749	0.5874	0.4191	0.6189	0.7805	0.6603	0.8950	0.9155	0.9100	0.9169	.9202
	COL	0.6454	0.6541	0.3679	0.6191	0.6237	0.4313	.6573	0.5962	0.5695	0.8760	0.9423	0.9174	0.9450	.9453
PLA	MOZ	0.6106	.6713	0.3839	0.6300	0.6485	0.3414	0.6545	0.7682	0.7230	0.8830	0.8142	0.8140	0.8060	.8149
	PLA	0.6699	0.6649	0.4139	0.6305	0.6352	0.4189	.6792	0.7293	0.6284	0.8850	0.8940	0.8858	0.8915	.8976
	POS	0.6443	.6863	0.3581	0.6316	0.6447	0.3743	0.6814	0.7402	0.5840	0.8910	0.9032	0.8943	0.9021	.9077
POS	BUG	0.6298	.6340	0.4538	0.6125	0.6108	0.4191	0.6328	0.5434	0.4492	0.8950	0.9142	0.8957	0.9169	.9187
	COL	0.6512	0.6490	0.3843	0.5985	0.6069	0.4313	.6544	0.4895	0.4936	0.8890	0.9422	0.9235	0.9450	.9450
	JDT	0.6352	0.6304	0.4174	0.5963	0.6011	0.4098	.6375	0.4846	0.4474	0.8970	0.8897	0.8668	0.8896	.8938
BUG	PLA	0.6354	.6548	0.4171	0.6247	0.6230	0.4189	0.6458	0.4976	0.4775	0.8990	0.8939	0.8767	0.8915	.8967
	POS	0.6656	.6831	0.3811	0.6397	0.6376	0.3743	0.6761	0.5250	0.4930	0.8990	0.9067	0.8924	0.9021	.9085
	BUG	0.6212	0.6301	0.4409	0.6043	0.6102	0.4191	.6352	0.6964	0.5822	0.8060	0.9149	0.9043	0.9169	.9201
COL	COL	0.6165	.6515	0.3679	0.6174	0.6215	0.4313	0.6324	0.5442	0.5964	0.7930	0.9433	0.9264	0.9450	.9455
	JDT	0.6527	0.6598	0.4075	0.6209	0.6256	0.4098	.6660	0.6238	0.5724	0.8030	0.8922	0.8754	0.8896	.8960
	MOZ	0.6397	.6944	0.3839	0.6507	0.6667	0.3414	0.6751	0.7183	0.6908	0.7990	0.8178	0.8164	0.8060	.8195
JDT	POS	0.6205	.6727	0.3581	0.6150	0.6382	0.3743	0.6683	0.6644	0.6120	0.8000	0.9037	0.8956	0.9021	.9062
	BUG	0.6194	.6296	0.4409	0.6038	0.6027	0.4191	0.6238	0.4984	0.3753	0.9020	0.9099	0.8755	.9169	0.9155
	COL	.6735	0.6501	0.3679	0.6151	0.6274	0.4313	0.6662	0.6368	0.6650	0.8840	0.9434	0.9348	.9450	0.9449
MOZ	JDT	0.6401	0.6412	0.4075	0.6113	0.6151	0.4098	.6510	0.5537	0.5426	.8970	0.8884	0.8750	0.8896	0.8914
	MOZ	0.6740	.7126	0.3839	0.6707	0.6819	0.3414	0.6938	0.5768	0.6111	.8940	0.8161	0.8056	0.8060	0.8149
	PLA	0.6360	0.6468	0.4139	0.6206	0.6219	0.4189	.6476	0.5563	0.5506	0.8980	0.8899	0.8751	0.8915	.8920
POS	AVG	0.6319	.6445	0.4083	0.6091	0.6147	0.3991	0.6434	0.6102	0.5427	0.8890	0.8920	0.8740	0.8919	.8956
	p -value	0.0029	0.6103	<.001	<.001	<.001	<.001	–	<.001	<.001	0.6103	<.001	<.001	<.001	–
	Cliff's δ	Small	Trivial	Large	Large	Large	Large	–	Large	Large	Trivial	Small	Medium	Small	–

naive bayes (NB) and decision tree (DT). By combining with original classifiers, we compare the performance of 26 different groups of EATT and employ the violin-plot to describe the distributions of the AUC and P_{opt} obtained from 10 times 10-fold cross validation at labeled rate 0.2. A violin plot plays a similar roles as a box and whisker plot and features a kernel density estimation of underlying distribution. For each model, the violin plot shows the median (the white dot), and the 25-th percentile (the lower side of black bold line), and the 75-th percentile (the upper side of the black bold line), and kernel density estimation (the external blue shape). Fig. 3 is obtained through combining the scores over six projects and the plots are ranked according to their medians.

From Fig. 3, we have the following observations. According to AUC, the diversity of base classifiers plays an important role in achieving promising performance. The models with same classifier are usually inferior to the ones with different classifiers (except SVM and LR). Meanwhile, the combination of strong classifiers usually has superior performance than weak ones. Particularly, there is large variance of the performance derived from three MLPs, while the problem could be alleviated by combining with other classifiers. The models involve RF, SVM, LR and MLP are usually better than that with NB and DT. This indicates that, it is of vital importance to train defect prediction model by choosing different and strong classifiers. According to P_{opt} , most combinations could achieve stable and high performance around 0.9 when predicting the order of changes (expect one case three DTs). With the proposed EATT framework, it does not matter which base classifier is selected for training, and different learners are enough to result in promising performance.

The above observations indicate that different base classifiers usually help EATT obtain preferable performance for JIT defect prediction. This kind of semi-supervised methods prefer to learners with a large disagreement such as multi-learners [43] or redundancy of feature rep-

resentation space [42]. As JIT defect datasets have only 14 features and two of them do not used, it is hard to meet the demands of redundant views. This explanation is in consistent with the result as Fig. 3 shows and may be the main reason why model trained with same classifier fail to obtain desirable performance. The diversity of base classifiers is usually considered as a characteristic of ensemble learning [59]. Studies have found that the disagreement between learners also contributes to semi-supervised methods by allocating pseudo-label to unlabeled data. This kind of semi-supervised learning method could be benefited from unlabeled data and multiple hypotheses at the same time [44].

Taken as a whole, above analysis indicates that the proposed EATT method can easily obtain stable and promising performance no matter whether predicting defect-inducing changes or defect-proneness of changes. Meanwhile, there are correlations between the scores of these two tasks (the model with outstanding performance in terms of AUC usually at the forefront in terms of P_{opt}). This setup allows us to focus energy on solving one problem and achieve performance gains for both types of tasks.

6.3. The effectiveness of semi-Supervised models with different assumptions

The prediction effectiveness of sample-based methods have been investigated exhaustively in previous sections. In this section, we further study the prediction effectiveness of other semi-supervised methods. Generally, semi-supervised learning method can be typically classified into four major categories, including generative-model based methods [60–62], low density separation based methods [63–66], graph-based methods [67–70] and disagreement-based methods [29,42–44]. In generative approaches, both labeled and unlabeled samples are assumed to be generated by the same parametric model. Methods in this category usually employ the EM (expectation-maximization) algo-

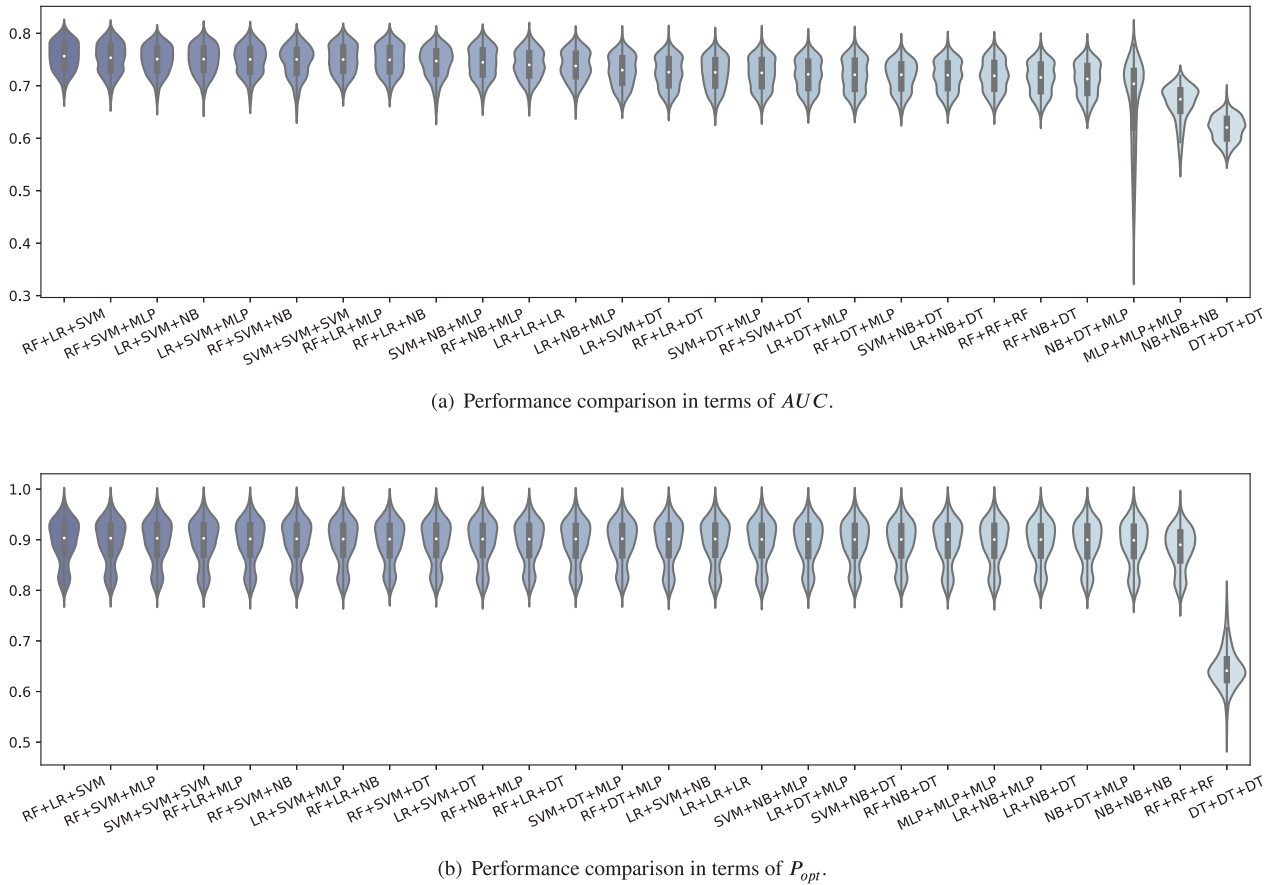


Fig. 3. Performance comparison with different combinations of base classifiers.

rithm [71] to conduct maximum likelihood estimation of the model parameters. We employ Gaussian maximum likelihood (GML) [62] to evaluate the performance of this kind of methods. Low density separation based methods try to use unlabeled data to adjust the decision boundary learned from the small number of labeled samples. Semi-supervised support vector machine (S3VM) [63] is used for special representative, which is one of the most popular methods in this category. Most graph-based learning usually focuses on conducting semi-supervised methods over a constructed graph, whose nodes are the training examples (both labeled and unlabeled) and the edges between nodes reflect certain relation (e.g. similarity). We use the manifold regularization based support vector machine (LapSVM) [68] to regularize the examples in a local region. Sample-based methods studied in this paper belong to disagreement-based semi-supervised learning, which uses multiple learners to exploit the disagreements among them during learning process. By teaching other learners with confident unlabeled samples, the generalization of learning algorithm could improve constantly. In order to verify the prediction effectiveness of these semi-supervised methods. We compare EATT with above typical methods based on different assumptions, including S3VM, LapSVM and GML under 10 times 10-fold cross validation at labeled rate 0.2.

Table 9 reports the average AUC and P_{opt} scores of the compared methods and presents the following observations. First, EATT could outperform other semi-supervised methods significantly in terms of AUC on all projects. There are five projects on which EATT achieve large effect sizes over the runner-up method (GML) and medium for the other models. These methods fail to obtain satisfactory performance as their assumption may not hold. All these approaches designed with directly or indirectly consideration, that there are clusters between positive (defect-inducing) and negative (defect-free) samples in our opinion. From Fig. 4, obviously, there is no clear characteristic of clusters between instances,

and the model built atop such assumption hard to improve performance with unlabeled data. Second, GML has similar performance as EATT in terms of P_{opt} while S3VM and LapSVM do not. We adopt same way (the proposed greedy strategy) to arrange their ranking order but inconsistent result appears between GML and the others. There are mainly two reasons why the performance of S3VM and LapSVM significantly worse than GML. First, the probabilities predicted by S3VM and LapSVM are of low confident, resulting in incorrect order and inferior performance when the rank list is inspected. Second, they are methods based on sup-

Table 9

The performance of the compared semi-supervised models with different assumptions.

	Project	S3VM	LapSVM	GML	EATT
AUC	BUG	.5866	.6365	.7129	.7408
	COL	.6609	.6244	.7098	.7310
	JDT	.6078	.5877	.7101	.7319
	MOZ	.6602	.6196	.7521	.7893
	PLA	.6315	.6044	.7335	.7603
	POS	.6699	.6413	.7581	.7796
	AVG	.6361	.6189	.7294	.7555
P_{opt}	BUG	.5000	.5942	.9211	.9247
	COL	.9345	.5655	.9343	.9349
	JDT	.6575	.5500	.8850	.8881
	MOZ	.6351	.5580	.8167	.8228
	PLA	.8899	.5629	.8976	.9005
	POS	.8963	.5498	.9010	.9057
	AVG	.7522	.5634	.8926	.8961

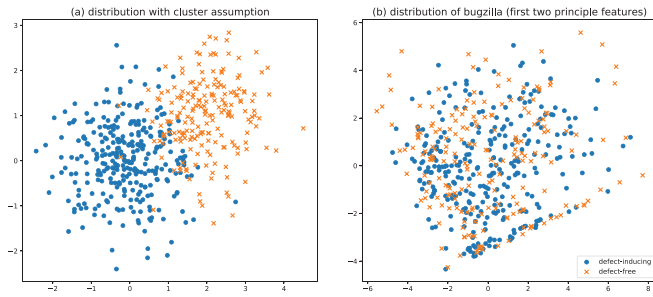


Fig. 4. Comparison of assumptive distribution (left) with real distribution (right).

port vector machine, which usually trained with labels represented by -1 and 1. This setting leads the results predicted by model scattered around 0, and hinders predictors from making wiser evaluation of risk value. More discussion could be found in Section 6.6.

Overall, the above analyses indicate EATT is a capable and appropriate semi-supervised method for JIT defect prediction. Methods of other categories need to be designed carefully to achieve better performance, and are worth further exploring.

6.4. Contribution of EATT over tri-Training model

In this paper, we propose effort-aware tri-training method, which is an extension of the traditional semi-supervised tri-training on the effort-aware defect prediction problem. In order to verify the effectiveness of our proposal, we have conducted a large number of experiments. However, the improvement of our proposed method compared to the tri-training method is still unknown. In this section, we will review EATT against non-effort-aware tri-training method [29] (TT) and a simple bagging-based supervised model [58] (BAG). The latter one is an ensemble learning method, which also adopted same three classifiers as others while the unlabeled instances are not used. We compared them under 10 times 10-folds cross-validation at labeled rate 0.2. Their average performance with respect to AUC and P_{opt} are reported in the following paragraph.

Table 10 summarizes the average AUC and P_{opt} for traditional tri-training method, bagging method and the proposed EATT. The following observations can be obtained. First, there is no significant difference between tri-training, bagging and EATT in terms of AUC . The average difference values are all less than 0.001, which indicates the unlabeled data makes no contribution to predicting the probabilities of defect-inducing changes. As the semi-supervised methods are more complicated and time-consuming for training, a much recommended scheme to make classification is to use supervised ones. Possible reasons have been revealed in Section 6.2 and Section 6.3. Second, according to P_{opt} , bagging and tri-training have a worse performance than EATT significantly. EATT achieves large magnitude of the difference over bagging and tri-training, the average improvements are 117.7% and 118.1%.

Table 10

The performance of the compared semi-supervised models.

Project	AUC			P_{opt}		
	BAG	TT	EATT	BAG	TT	EATT
BUG	.7401	.7390	.7408	.5630	.5617	.9247
COL	.7323	.7327	.7310	.2741	.2698	.9349
JDT	.7319	.7324	.7319	.4068	.4067	.8881
MOZ	.7897	.7895	.7893	.4573	.4572	.8228
PLA	.7604	.7604	.7603	.4143	.4136	.9005
POS	.7792	.7791	.7796	.3544	.3555	.9057
AVG	.7556	.7555	.7555	.4116	.4107	.8961

Overall, the above analysis suggests that the proposed effort-aware strategy could improve traditional non-effort-aware methods greatly and help find more defect-inducing changes with less inspect effort.

6.5. Performance at different labeled rates

We only report experimental results at labeled rates of 10% and 20% in the preceding sections, and the reliance of our model on supervised information is not fully revealed. It is worth investigating the trend of prediction performance as the software development process evolves. This study can provide an intuitive perspective on how the labeled data contributes to the prediction performance in real-world application scenarios. To this end, we further compare EATT with the supervised CBS model, unsupervised CCUM model and semi-supervised FTF model at various values of the labeled rate from 1% to 100% with the step size of 1%. We choose CBS, CCUM and FTF because their promising performance are examined in previous sections. As Fig. 6 shows, our model can achieve satisfactory results and outperform most of baseline models (except CCUM in case of project Columba) significantly with only 10% of labeled data available in the ranking scenario. This indicates that software engineers can allocate the available testing resources more effectively in the early phases of development lifecycle.

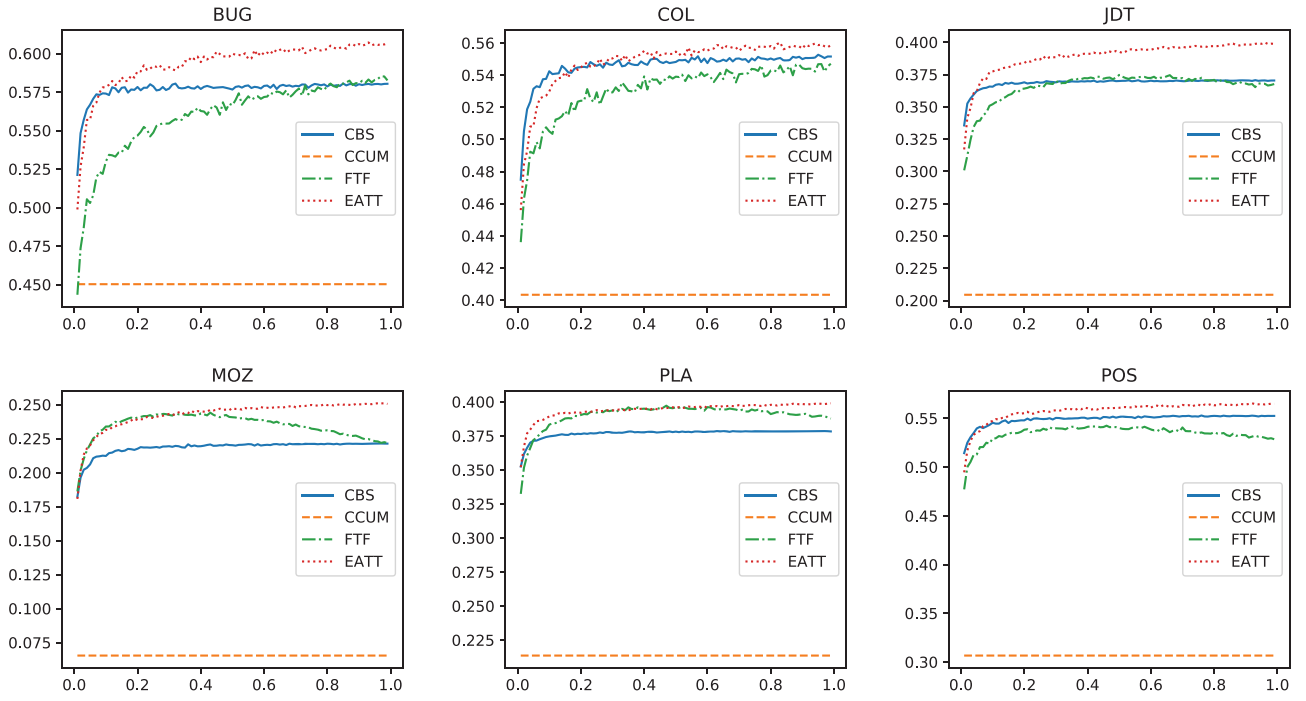
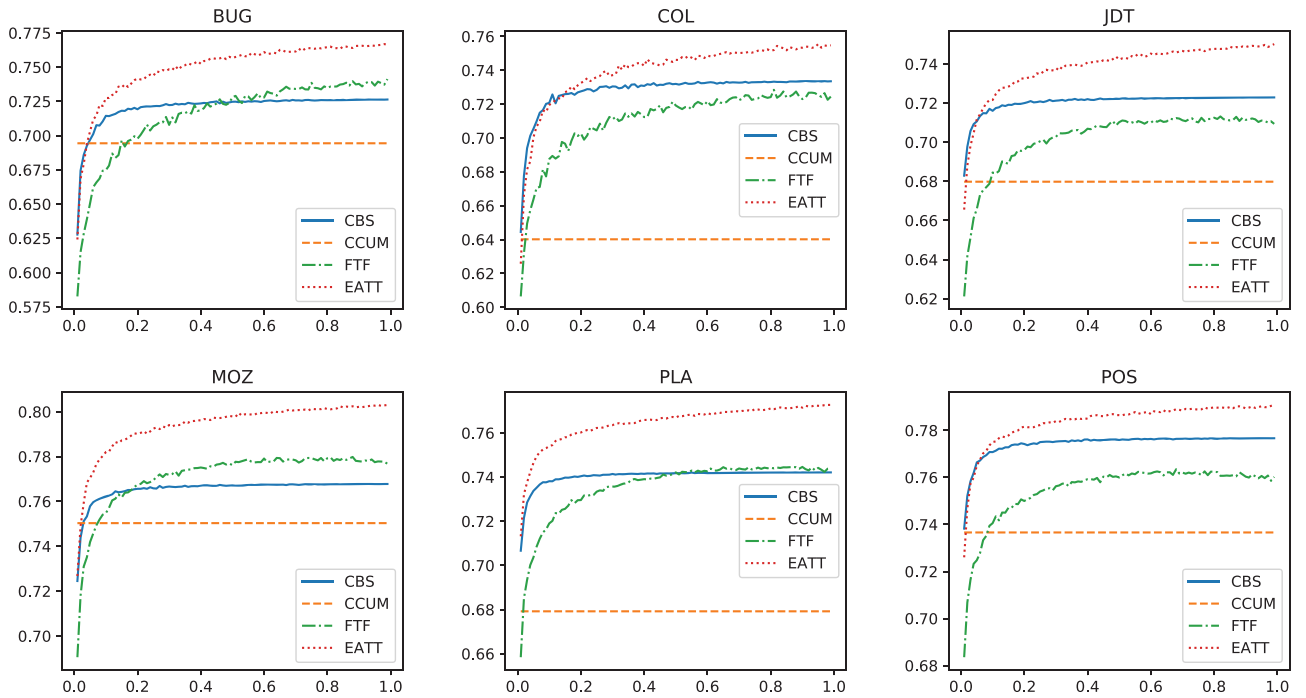
At the same time, the effect of various values of the labeled rate on classification performance is also studied in Fig. 5. There is no doubt that supervised information has a substantial influence on the classification results, as the unsupervised CCUM model performs poorly for all datasets. A small decrease of $F1$ and AUC is observed on the larger datasets (JDT, Mozilla, Platform and PostgreSQL) as the rate of labeled data increases for the FTF model. An increasing advantage of EATT can be observed according to AUC as the labeled rate gains. As only three baseline models are used, we will not go on appraising the performance of EATT under the supervised JIT defect prediction.

Overall, above results demonstrate that our semi-supervised model has a large advantage over other models if the labeled data is insufficient.

6.6. Factors for building a good effort-Aware defect prediction model

One reason many supervised models fail to outperform unsupervised ones is that the former gives preference to the changes predicted to be defect-inducing. The corresponding ranked list manifests a higher priority assigned to the predicted defect-inducing changes rather than changes with smaller “size”, which is the major obstacle to making good predictions (e.g., in case of CBS). An influential factor in the evaluation of an effort-aware model is the amount of effort required to review a change, referred to as LOC in the previous section. As observed from Fig. 7, the distribution of LOC is highly skewed, which results in most of the review budget being taken up by a small number of changes. A similar observation was reported in Ref. [35], while the effect on performance remained unclear. In Fig. 8, nearly 20% of effort is expended by CBS in the range of [0.4, 0.6], while few defect-inducing changes are detected. A model has less chance to detect more defect-inducing changes if it is applied prior to a review of large-scale changes. Therefore, a more appropriate method would use the risk value predicted by the model as a priori knowledge, and then consider this problem as the *knapsack problem* to arrange a more efficient review order of changes.

Another important factor that affects performance significantly is the accuracy of the risk value estimated by models. Models based on a single classifier require a large quantity of labeled data to achieve good performance (e.g., FTF requires more than 50% of labeled samples to achieve performance similar to that of CCUM). However, the ranking results can be significantly improved by combining the diversity information of multiple individual learners, even if only a small number of labeled samples are available. This is also a straightforward explanation of robustness of our model.

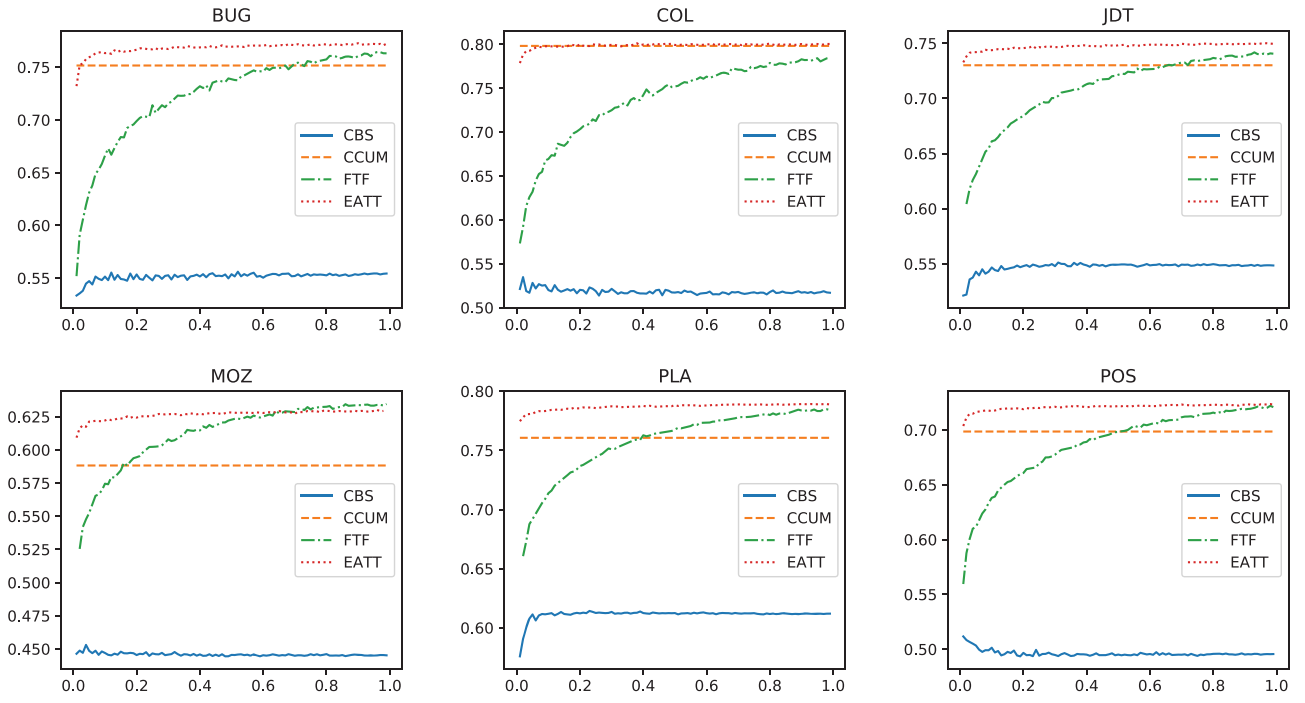
(a) Performance comparison in terms of $F1$.(b) Performance comparison in terms of AUC .**Fig. 5.** Classification performance comparison at different labeled rates.

7. Threats to validity

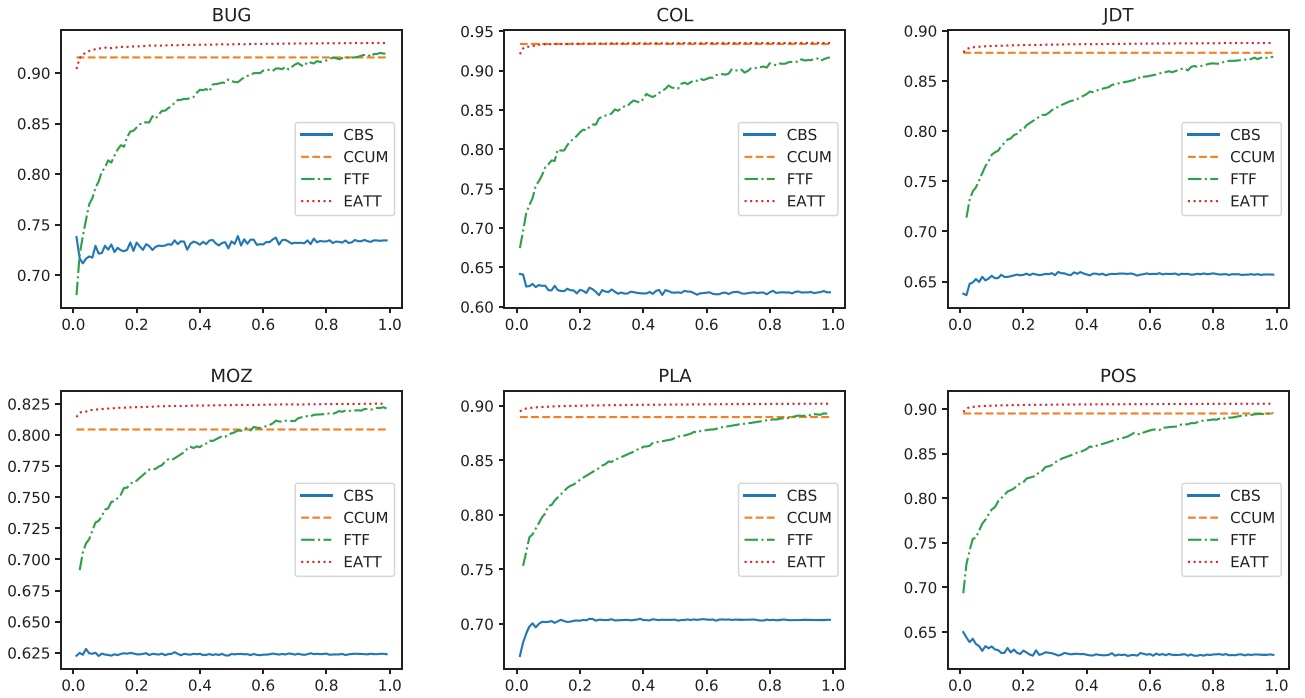
The concept of threats to validity refers to the difference between what a research study actually explores and what the researcher purports to investigate [72]. To address the observed potential limitations of our findings, the following discusses the threats to the construct, internal and external validity of the study, as recommended by Wohlin et al. [73].

Construct validity is the degree to which the dependent and independent variables accurately measure the concept they purport to measure.

The concept of threats to validity refers to the difference between what a research study actually explores and what the researcher purports to investigate [72]. To address the observed potential limitations of our findings, the following discusses the threats to the construct, internal and external validity of the study, as recommended by Wohlin et al. [73].



(a) Performance comparison in terms of ACC.

(b) Performance comparison in terms of P_{opt} .**Fig. 6.** Ranking performance comparison at different labeled rates.

Our study uses the same datasets that were used in previous effort-aware JIT defect prediction studies. The defect-inducing changes are discovered by the commonly used SZZ algorithm [30], and may be incomplete. Furthermore, the measure of the effort required to review a change is considered to be the total number of lines modified by that

change, which may be inconsistent with that in the real world. However, these threats represent problems inherent to most studies and need to be further explored by further research.

The independent variables used in this study are commonly used change metrics, and the degree to which they accurately measure the

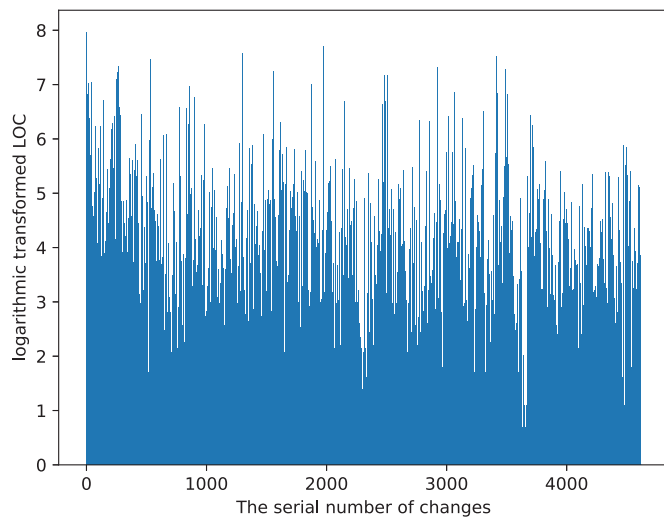


Fig. 7. The effort values of bugzilla. (The ordinate value is logarithmically transformed.).

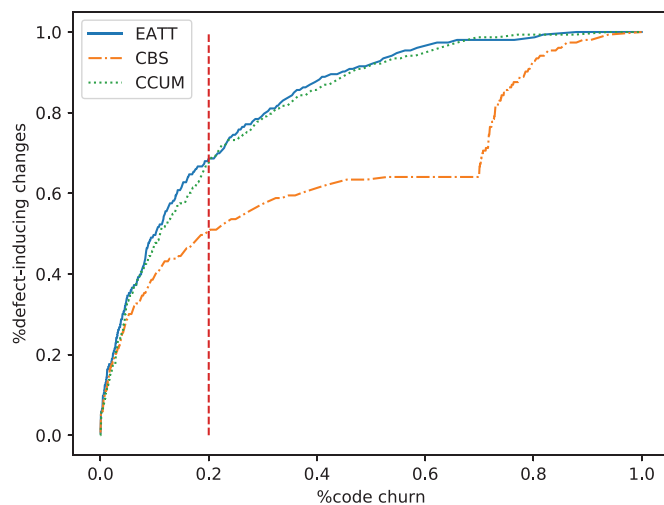


Fig. 8. Example of an effort-based cumulative lift chart for bugzilla.

concepts they purport to measure has been investigated. The construct validity of the independent variable is acceptable in this study.

Internal validity is the degree to which conclusions can be drawn about the causal effect of independent variables on the dependent variables.

As we try to study semi-supervised JIT defect prediction, the selection of labeled data will have a large influence on the results while these data are distributed completely randomly. At the same time, the choice of labeled rates may lead to different conclusions. To alleviate these threats, we performed experiments at various labeled rates under 10 times 10-fold cross-validation. The results reported in Section 5 and Section 6 are the average values of all 100 prediction scores. The bias of evaluation could be considerably reduced by using this setting.

Another threat is the unknown effect of the selection of classifiers. Our study used three different learners (namely, logistic regression, support vector machine and random forest) as base models of our semi-supervised method. The effectiveness of other classifiers remains unverified and needs to be studied in the future.

External validity is the degree to which the results of research can be generalized to the population under study and other research settings.

Six long-lived and widely used open-source software systems considered in our experiments are large enough to allow drawing statistically

meaningful conclusions. Our results may not be generalizable to other systems that have features different from those of the studied datasets. Furthermore, the distribution of effort value may not be the same as that reported in Section 6.6, which may result in reaching conclusions different from ours. It is necessary to consider a wide variety of systems to replicate our study in the future to mitigate these threats.

8. Conclusion

In this paper, we investigate the predictive effectiveness of semi-supervised learning for effort-aware JIT defect prediction. To this end, we propose an effort-aware tri-training (EATT) method. Extensive comparison experiments are performed to demonstrate the effectiveness of EATT. The experimental results on six large projects show that EATT has a considerable advantage over supervised and unsupervised models. On the one hand, EATT can significantly improve the ability to detect defect-inducing changes while ensuring high prediction accuracy. It outperforms almost all supervised and unsupervised models in the ranking scenario and is comparable to supervised models in the classification scenario. On the other hand, EATT retains the advantage of unsupervised models, requiring only a small quantity of labeled data for training. We further investigate the influence of labeled rate and clearly demonstrate the effectiveness of our semi-supervised model even if the labeled data is insufficient. Therefore, this study has shown that semi-supervised learning is more desirable for practical applications, since it can effectively combine the advantages of supervised and unsupervised learning.

CRedit authorship contribution statement

Weiwei Li: Conceptualization, Methodology, Writing - original draft, Writing - review & editing. **Wenzhou Zhang:** Software, Validation, Investigation, Writing - original draft. **Xiuyi Jia:** Investigation, Supervision. **Zhiqiu Huang:** Project administration.

Acknowledgements

This work is jointly supported by the National Key R&D Program of China (No. 2018YFB1003902), the [Natural Science Foundation of Jiangsu Province](#) (Nos. BK20170809, BK20191287), the [National Natural Science Foundation of China](#) (Nos. 61906090, 61773208), and the [Fundamental Research Funds for the Central Universities](#) (No. 30920021131).

References

- [1] T. Hall, S. Beecham, D. Bowes, D. Gray, S. Counsell, A systematic literature review on fault prediction performance in software engineering, *IEEE Trans. Softw. Eng.* 38 (6) (2012) 1276–1304.
- [2] A. Mockus, D.M. Weiss, Predicting risk of software changes, *Bell Labs Tech. J.* 5 (2) (2000) 169–180.
- [3] Y. Kamei, E. Shihab, B. Adams, A.E. Hassan, A. Mockus, A. Sinha, N. Ubayashi, A large-scale empirical study of just-in-time quality assurance, *IEEE Trans. Softw. Eng.* 39 (6) (2013) 757–773.
- [4] T. Gyimóthy, R. Ferenc, I. Siket, Empirical validation of object-oriented metrics on open source software for fault prediction, *IEEE Trans. Softw. Eng.* 31 (10) (2005) 897–910.
- [5] A.E. Hassan, Predicting faults using the complexity of code changes, in: 31st International Conference on Software Engineering, ICSE 2009, May 16–24, 2009, Vancouver, Canada, Proceedings, 2009, pp. 78–88.
- [6] P.L. Li, J.D. Herbsleb, M. Shaw, B. Robinson, Experiences and results from initiating field defect prediction and product test prioritization efforts at ABB inc, in: 28th International Conference on Software Engineering (ICSE 2006), Shanghai, China, May 20–28, 2006, 2006, pp. 413–422.
- [7] J.C. Munson, T.M. Khoshgoftaar, The detection of fault-prone programs, *IEEE Trans. Softw. Eng.* 18 (5) (1992) 423–433.
- [8] Q. Huang, X. Xia, D. Lo, Revisiting supervised and unsupervised models for effort-aware just-in-time defect prediction, *Empir. Softw. Eng.* 24 (5) (2019) 2823–2862.
- [9] X. Yang, D. Lo, X. Xia, J. Sun, TLEL: a two-layer ensemble learning approach for just-in-time defect prediction, *Inf. Softw. Technol.* 87 (2017) 206–220.

- [10] S. Young, T. Abdou, A. Bener, A replication study: just-in-time defect prediction with ensemble learning, in: 6th IEEE/ACM International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering, RAISE@ICSE 2018, Gothenburg, Sweden, May 27, 2018, 2018, pp. 42–47.
- [11] X. Chen, Y. Zhao, Q. Wang, Z. Yuan, MULTI: Multi-objective effort-aware just-in-time software defect prediction, *Inf. Software. Technol.* 93 (2018) 1–13.
- [12] W. Fu, T. Menzies, Revisiting unsupervised learning for defect prediction, in: Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017, Paderborn, Germany, September 4–8, 2017, 2017, pp. 72–83.
- [13] Y. Yang, Y. Zhou, J. Liu, Y. Zhao, H. Lu, L. Xu, B. Xu, H. Leung, Effort-aware just-in-time defect prediction: simple unsupervised models could be better than supervised models, in: Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2016, Seattle, WA, USA, November 13–18, 2016, 2016, pp. 157–168.
- [14] O. Chapelle, B. Scholkopf, E. A. Zien, Semi-supervised learning, *IEEE Transactions on Neural Networks* 20 (3) (2009). 542–542.
- [15] X. Zhu, Semi-supervised Learning, in: Encyclopedia of Machine Learning and Data Mining, Springer, 2017, pp. 1142–1147.
- [16] A. Arshad, S. Riaz, L. Jiao, A. Murthy, Semi-supervised deep fuzzy c-mean clustering for software fault prediction, *IEEE Access* 6 (2018) 25675–25685.
- [17] C. Catal, B. Diri, Unlabelled extra data do not always mean extra performance for semi-supervised fault prediction, *Expert Syst.* 26 (5) (2009) 458–471.
- [18] Q. He, B. Shen, Y. Chen, Software defect prediction using semi-supervised learning with change burst information, in: 40th IEEE Annual Computer Software and Applications Conference, COMPSAC 2016, Atlanta, GA, USA, June 10–14, 2016, IEEE Computer Society, 2016, pp. 113–122.
- [19] Y. Jiang, M. Li, Z. Zhou, Software defect detection with rocus, *J. Comput. Sci. Technol.* 26 (2) (2011) 328–342.
- [20] M. Li, H. Zhang, R. Wu, Z. Zhou, Sample-based software defect prediction with active and semi-supervised learning, *Automat. Softw. Eng.* 19 (2) (2012) 201–230.
- [21] H. Lu, B. Cukic, M.V. Culp, An iterative semi-supervised approach to software fault prediction, in: Proceedings of the 7th International Conference on Predictive Models in Software Engineering, PROMISE 2011, Banff, Alberta, Canada, September 20–21, 2011, 2011, p. 15.
- [22] H. Lu, B. Cukic, M.V. Culp, Software defect prediction using semi-supervised learning with dimension reduction, in: IEEE/ACM International Conference on Automated Software Engineering, ASE'12, Essen, Germany, September 3–7, 2012, 2012, pp. 314–317.
- [23] H. Lu, B. Cukic, M.V. Culp, A semi-supervised approach to software defect prediction, in: IEEE 38th Annual Computer Software and Applications Conference, COMPSAC 2014, Vasteras, Sweden, July 21–25, 2014, IEEE Computer Society, 2014, pp. 416–425.
- [24] Y. Ma, W. Pan, S. Zhu, H. Yin, J. Luo, An improved semi-supervised learning method for software defect prediction, *J. Intell. Fuzzy Syst.* 27 (5) (2014) 2473–2480.
- [25] N. Seliya, T.M. Khoshgoftaar, Software quality analysis of unlabeled program modules with semisupervised clustering, *IEEE Trans. Syst. Man Cybernet. Part A* 37 (2) (2007) 201–211.
- [26] N. Seliya, T.M. Khoshgoftaar, Software quality estimation with limited fault data: a semi-supervised learning perspective, *Softw. Q. J.* 15 (3) (2007) 327–344.
- [27] Z. Zhang, X. Jing, T. Wang, Label propagation based semi-supervised learning for software defect prediction, *Automat. Softw. Eng.* 24 (1) (2017) 47–69.
- [28] A. Syed, Investigating the practicality of just-in-time defect prediction with semi-supervised learning on industrial commit data, 2019.
- [29] Z. Zhou, M. Li, Tri-training: exploiting unlabeled data using three classifiers, *IEEE Trans. Know. Data Eng.* 17 (11) (2005) 1529–1541.
- [30] J. Sliwerski, T. Zimmermann, A. Zeller, When do changes induce fixes? *ACM SIGSOFT Softw. Eng. Notes* 30 (4) (2005) 1–5.
- [31] J. Eyoifson, L. Tan, P. Lam, Do time of day and developer experience affect commit bugginess, in: Proceedings of the 8th International Working Conference on Mining Software Repositories, MSR 2011 (Co-located with ICSE), Waikiki, Honolulu, HI, USA, May 21–28, 2011, Proceedings, 2011, pp. 153–162.
- [32] Z. Yin, D. Yuan, Y. Zhou, S. Pasupathy, L.N. Bairavasundaram, How do fixes become bugs? in: SIGSOFT/FSE'11 19th ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE-19) and ESEC'11: 13th European Software Engineering Conference (ESEC-13), Szeged, Hungary, September 5–9, 2011, 2011, pp. 26–36.
- [33] S. Kim, E.J.W. Jr., Y. Zhang, Classifying software changes: clean or buggy? *IEEE Trans. Softw. Eng.* 34 (2) (2008) 181–196.
- [34] Y. Kamei, T. Fukushima, S. McIntosh, K. Yamashita, N. Ubayashi, A.E. Hassan, Studying just-in-time defect prediction using cross-project models, *Empir. Softw. Eng.* 21 (5) (2016) 2072–2106.
- [35] Q. Huang, X. Xia, D. Lo, Supervised vs unsupervised models: a holistic look at effort-aware just-in-time defect prediction, in: 2017 IEEE International Conference on Software Maintenance and Evolution, ICSME 2017, Shanghai, China, September 17–22, 2017, 2017, pp. 159–170.
- [36] X. Yang, D. Lo, X. Xia, Y. Zhang, J. Sun, Deep learning for just-in-time defect prediction, in: 2015 IEEE International Conference on Software Quality, Reliability and Security, QRS 2015, Vancouver, BC, Canada, August 3–5, 2015, 2015, pp. 17–26.
- [37] J. Liu, Y. Zhou, Y. Yang, H. Lu, B. Xu, Code churn: a neglected metric in effort-aware just-in-time defect prediction, in: 2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM 2017, Toronto, ON, Canada, November 9–10, 2017, 2017, pp. 11–19.
- [38] L. Qiao, Y. Wang, Effort-aware and just-in-time defect prediction with neural network, *PLoS ONE* 14 (2) (2019) e0211359.
- [39] G. Catolino, D.D. Nucci, F. Ferrucci, Cross-project just-in-time bug prediction for mobile apps: an empirical assessment, in: Proceedings of the 6th International Conference on Mobile Software Engineering and Systems, MOBILESoft@ICSE 2019, Montreal, QC, Canada, May 25, 2019, 2019, pp. 99–110.
- [40] M. Kondo, D.M. German, O. Mizuno, E.-H. Choi, The impact of context metrics on just-in-time defect prediction, *Empir. Softw. Eng.* (2019) 1–50.
- [41] Y. Fan, X. Xia, D.A. da Costa, D. Lo, A.E. Hassan, S. Li, The impact of changes mislabeled by szz on just-in-time defect prediction, *IEEE Trans. Softw. Eng.* (2019).
- [42] A. Blum, T.M. Mitchell, Combining labeled and unlabeled data with co-training, in: Proceedings of the Eleventh Annual Conference on Computational Learning Theory, COLT 1998, Madison, Wisconsin, USA, July 24–26, 1998., 1998, pp. 92–100.
- [43] S.A. Goldman, Y. Zhou, Enhancing supervised learning with unlabeled data, in: Proceedings of the Seventeenth International Conference on Machine Learning (ICML 2000), Stanford University, Stanford, CA, USA, June 29, - July 2, 2000, 2000, pp. 327–334.
- [44] Z. Zhou, M. Li, Semi-supervised learning by disagreement, *Knowl. Inf. Syst.* 24 (3) (2010) 415–439.
- [45] N. Nagappan, T. Ball, A. Zeller, Mining metrics to predict component failures, in: 28th International Conference on Software Engineering (ICSE 2006), Shanghai, China, May 20–28, 2006, 2006, pp. 452–461.
- [46] M. D'Ambros, M. Lanza, R. Robbes, An extensive comparison of bug prediction approaches, in: Proceedings of the 7th International Working Conference on Mining Software Repositories, MSR 2010 (Co-located with ICSE), Cape Town, South Africa, May 2–3, 2010, Proceedings, 2010, pp. 31–41.
- [47] R. Moser, W. Pedrycz, G. Succi, A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction, in: 30th International Conference on Software Engineering (ICSE 2008), Leipzig, Germany, May 10–18, 2008, 2008, pp. 181–190.
- [48] N. Nagappan, T. Ball, Use of relative code churn measures to predict system defect density, in: 27th International Conference on Software Engineering (ICSE 2005), 15–21 May 2005, St. Louis, Missouri, USA, 2005, pp. 284–292.
- [49] A.G. Koru, D. Zhang, K.E. Emam, H. Liu, An investigation into the functional form of the size-defect relationship for software modules, *IEEE Trans. Softw. Eng.* 35 (2) (2009) 293–304.
- [50] P.J. Guo, T. Zimmermann, N. Nagappan, B. Murphy, Characterizing and predicting which bugs get fixed: an empirical study of microsoft windows, in: Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering - Volume 1, ICSE 2010, Cape Town, South Africa, 1–8 May 2010, 2010, pp. 495–504.
- [51] R. Purushothaman, D.E. Perry, Toward understanding the rhetoric of small source code changes, *IEEE Trans. Software Eng.* 31 (6) (2005) 511–526.
- [52] S. Matsumoto, Y. Kamei, A. Monden, K. Matsumoto, M. Nakamura, An analysis of developer metrics for fault prediction, in: Proceedings of the 6th International Conference on Predictive Models in Software Engineering, PROMISE 2010, Timisoara, Romania, September 12–13, 2010, 2010, p. 18.
- [53] T.L. Graves, A.F. Karr, J.S. Marron, H.P. Siy, Predicting fault incidence using software change history, *IEEE Trans. Softw. Eng.* 26 (7) (2000) 653–661.
- [54] D. Angluin, P.D. Laird, Learning from noisy examples, *Mach. Learn.* 2 (4) (1987) 343–370.
- [55] F. Wilcoxon, Individual comparisons by ranking methods, in: Breakthroughs in Statistics, Springer, 1992, pp. 196–202.
- [56] Y. Benjamini, Y. Hochberg, Controlling the false discovery rate: a practical and powerful approach to multiple testing, *J. R. Stat. Soc.* 57 (1) (1995) 289–300.
- [57] R. Jeanine, K.J. D. C. Jesse, S. Jeff, D. Linda, Exploring methods for evaluating group differences on the NSSE and other surveys: Are the t-test and cohens d indices the most appropriate choices, in: Annual Meeting of the Southern Association for Institutional Research, 2006, pp. 1–51.
- [58] L. Breiman, Bagging predictors, *Mach. Learn.* 24 (2) (1996) 123–140.
- [59] C. Zhang, Y. Ma, Ensemble machine learning: Methods and applications, Springer, 2012.
- [60] D.J. Miller, H.S. Uyar, A mixture of experts classifier with learning based on both labelled and unlabelled data, in: Advances in Neural Information Processing Systems 9, NIPS, Denver, CO, USA, December 2–5, 1996, 1996, pp. 571–577.
- [61] K. Nigam, A. McCallum, S. Thrun, T.M. Mitchell, Text classification from labeled and unlabeled documents using EM, *Mach. Learn.* 39 (2/3) (2000) 103–134.
- [62] B.M. Shahshahani, D.A. Landgrebe, The effect of unlabeled samples in reducing the small sample size problem and mitigating the hughes phenomenon, *IEEE Trans. Geosci. Remote Sens.* 32 (5) (1994) 1087–1095.
- [63] O. Chapelle, A. Zien, Semi-supervised classification by low density separation, in: Proceedings of the Tenth International Workshop on Artificial Intelligence and Statistics, AISTATS 2005, Bridgetown, Barbados, January 6–8, 2005, 2005, 2005, pp. 57–64.
- [64] Y. Grandvalet, Y. Bengio, Semi-supervised learning by entropy minimization, in: Advances in Neural Information Processing Systems 17 Neural Information Processing Systems, NIPS 2004, December 13–18, 2004, Vancouver, British Columbia, Canada, 2004, pp. 529–536.
- [65] T. Joachims, Transductive inference for text classification using support vector machines, in: Proceedings of the Sixteenth International Conference on Machine Learning (ICML 1999), Bled, Slovenia, June 27, - 30, 1999, 1999, pp. 200–209.
- [66] N.D. Lawrence, M.I. Jordan, Semi-supervised learning via gaussian processes, in: Advances in Neural Information Processing Systems 17 [Neural Information Processing Systems, NIPS 2004, December 13–18, 2004, Vancouver, British Columbia, Canada], 2004, pp. 753–760.
- [67] M. Belkin, P. Niyogi, Semi-supervised learning on riemannian manifolds, *Mach. Learn.* 56 (1–3) (2004) 209–239.
- [68] M. Belkin, P. Niyogi, V. Sindhwani, Manifold regularization: a geometric framework for learning from labeled and unlabeled examples, *J. Mach. Learn. Res.* 7 (2006) 2399–2434.

- [69] D. Zhou, O. Bousquet, T.N. Lal, J. Weston, B. Schölkopf, Learning with local and global consistency, in: Advances in Neural Information Processing Systems 16 [Neural Information Processing Systems, NIPS 2003, December 8–13, 2003, Vancouver and Whistler, British Columbia, Canada], 2003, pp. 321–328.
- [70] X. Zhu, Z. Ghahramani, J.D. Lafferty, Semi-supervised learning using gaussian fields and harmonic functions, in: Machine Learning, Proceedings of the Twentieth International Conference (ICML 2003), August 21–24, 2003, Washington, DC, USA, 2003, pp. 912–919.
- [71] A.P. Dempster, N.M. Laird, D.B. Rubin, Maximum likelihood from incomplete data via the em algorithm, *J. R. Stat. Soc.* 39 (1) (1977) 1–22.
- [72] J. Siegmund, N. Siegmund, S. Apel, Views on internal and external validity in empirical software engineering, in: 37th IEEE/ACM International Conference on Software Engineering, ICSE 2015, Florence, Italy, May 16–24, 2015, Volume 1, 2015, pp. 9–19.
- [73] C. Wohlin, P. Runeson, M. Höst, M.C. Ohlsson, B. Regnell, Experimentation in Software Engineering, Springer, 2012.